

## 课程笔记

# AI-12 Agent 一线实战：落地经验与 Know-How

基于 Bilibili 讲座内容整理

**KCD Beijing + vLLM 2026**

### Agent 框架本身价值较低

- 类 langchain/langgraph 框架，希望在 llm 和应用之间构建一层抽象，但提供的价值较低
- Agent 本身就是一个大的 while 循环，研发成本较低，在 vibe coding 成熟下，成本更低。模型看状态 → 决定下一步 → 执行工具或回答 → 循环
- LLM 对全世界来说都是新的，OpenAI 和 Anthropic 并不掌握更多

主办方: KCD BEIJING, vLLM, OPEN NATIVE  
特别支持方: 亚马逊云科技, HUAWEI, Red Hat, Valkey, Akamai  
钻石赞助商: 亚马逊云科技  
白金赞助商: HUAWEI, Red Hat  
黄金赞助商: Valkey, Akamai

视频作者 / 频道: KCD-China

发布时间: 2026-04-08

视频时长: 38:47

视频链接: <https://www.bilibili.com/video/BV13ZDLB4EW5/>

# 目录

|                                                  |           |
|--------------------------------------------------|-----------|
| <b>1 Agent 的真实壁垒：一线 Know-How</b>                 | <b>3</b>  |
| 1.1 开场：这场分享要回答什么                                 | 3         |
| 1.2 Agent：一个由模型驱动的循环                             | 3         |
| 1.3 框架抽象的价值为什么会变薄                                | 3         |
| 1.4 真正的壁垒：在一线发现模型边界                              | 4         |
| 1.5 发现问题本身就是高价值资产                                | 4         |
| 1.6 从 S1 走向 S2：第一条具体 Know-How 是上下文               | 5         |
| 1.7 本章小结                                         | 5         |
| <b>2 上下文是一切：从 Harmony 到缓存命中</b>                  | <b>5</b>  |
| 2.1 为什么上下文比代码更接近 Agent 的核心                       | 5         |
| 2.2 OpenAI Harmony：所谓“魔法”会被序列化成本                 | 6         |
| 2.3 上下文构造：把什么放进去，比调用什么 API 更要命                   | 7         |
| 2.4 Prompt cache：大规模 Agent 的成本纪律                 | 7         |
| 2.5 Prefill：为什么缓存命中也会影响延迟                        | 7         |
| 2.6 哪些动态信息最容易打碎缓存                                | 8         |
| 2.7 从 S2 走向 S3：缓存纪律之后是评测纪律                       | 8         |
| 2.8 本章小结                                         | 8         |
| <b>3 模型切换与评测基线：别相信官方跑分</b>                       | <b>8</b>  |
| 3.1 模型切换的成本在哪里                                   | 10        |
| 3.2 先搭 demo，再建立应用 Benchmark                      | 10        |
| 3.3 用轻量符号描述应用基线                                  | 11        |
| 3.4 新模型、降本模型，都要过同一把尺                             | 12        |
| 3.5 长上下文退化：越塞越多不等于越懂                             | 12        |
| 3.6 本章小结                                         | 13        |
| <b>4 上下文管理的工程难题：检索、错误、动态提示与压缩</b>                | <b>14</b> |
| 4.1 RAG 的问题：答案以外，搜索过程也在教模型                       | 14        |
| 4.2 失败工具调用：删掉省 token，留下防复读                       | 15        |
| 4.3 动态提示注入：用规则提醒智能体的悖论                           | 15        |
| 4.4 Few-shot 与 prompt perturbation：别让模型偷懒复制      | 16        |
| 4.5 Compaction 与幻觉累积：代码压缩尤其危险                    | 16        |
| 4.6 本章小结                                         | 17        |
| <b>5 多 Agent 协作：Subagent 的优雅与 Agent Team 的风险</b> | <b>19</b> |
| 5.1 从 router 设计说起：把请求分发给专门角色                     | 19        |
| 5.2 Subagent 模式：隔离任务，减少过程沟通                      | 20        |
| 5.3 summary bottleneck：多 Agent 沟通的真实瓶颈           | 20        |
| 5.4 Agent Team：看起来更像组织，也更容易积累幻觉                  | 21        |
| 5.5 实践判断：什么时候用 Subagent，什么时候警惕 Agent Team        | 21        |
| 5.6 本章小结                                         | 22        |

|                                                 |           |
|-------------------------------------------------|-----------|
| <b>6 Bitter Lesson for Agents: 模型变强时，架构要变简单</b> | <b>22</b> |
| 6.1 Bitter Lesson: 复杂技巧的有效期                     | 23        |
| 6.2 base model capability 决定 Agent architecture | 23        |
| 6.3 人类预定义范式要适时退场                                | 24        |
| 6.4 Chain-of-Thought 可能只是训练轮                    | 24        |
| 6.5 回到第一性原理：让模型自由观察、试错、反馈                       | 25        |
| 6.6 ReAct: 短期内最难被拿掉的核心循环                        | 25        |
| 6.7 本章小结                                        | 25        |
| <b>7 工具设计与 CLI 趋势：工具也是提示词</b>                   | <b>27</b> |
| 7.1 工具回传也是上下文：没有高低贵贱                            | 27        |
| 7.2 工具名、描述和参数名都在写 prompt                        | 28        |
| 7.3 把结构化输出伪装成工具调用                               | 28        |
| 7.4 不要标新立异：模型也吃“通识”                             | 29        |
| 7.5 工具数量、MCP 与缓存压力                              | 29        |
| 7.6 CLI 的优势：渐进式披露                               | 30        |
| 7.7 本章小结                                        | 30        |
| <b>8 Memory 与市场教育：长程智能体如何被接受</b>                | <b>31</b> |
| 8.1 Memory 的三道难题：抽取、检索、更新                       | 31        |
| 8.2 全量上下文筛选与语义检索的取舍                             | 31        |
| 8.3 system prompt 里的记忆会过度激活                     | 32        |
| 8.4 memory.md 与 JSON memory                     | 32        |
| 8.5 市场教育：用户开始接受看结果                              | 33        |
| 8.6 长程智能体的信任边界                                  | 33        |
| 8.7 本章小结                                        | 33        |
| <b>9 最终综合：Agent 工程的三条主线</b>                     | <b>35</b> |
| 9.1 第一条主线：一切进入模型的东西都是上下文                        | 35        |
| 9.2 第二条主线：没有应用 benchmark，就没有真实改进                | 35        |
| 9.3 第三条主线：模型变强时，架构要变简单                          | 36        |
| 9.4 一个耦合系统：成本、延迟、工具、记忆与信任                       | 36        |
| 9.5 市场教育完成后，工程门槛更高                              | 37        |

# 1 Agent 的真实壁垒：一线 Know-How

## 1.1 开场：这场分享要回答什么

这场分享从一个很务实的问题开始：今天做 Agent，真正有价值的经验到底在哪里？讲者张开翼先交了自己的项目背景：当前参与 Kimi Code CLI，之前在微软参与过 Prompt Flow（据 ASR 上下文校正），也参与过 GitHub Copilot Agent，以及与 OpenAI 相关的模型后训练和 Agent benchmark 工作。这个背景重要，因为后面所有判断都来自一线产品试错，并且落到框架层面之外的真实工程细节。

这里还要补上一层定位：Kimi Code CLI 带有模型厂商视角，产品团队可以和底层模型一起联调，把下一代 coding policy、工具使用习惯和模型能力边界的判断写进产品。这个前提会在后文回到架构问题上：当模型侧能持续训练工具行为，Agent 外部脚手架的最优形态也会跟着变化。

他给这场分享定下的题眼是“落地经验与 Know-How”。这里的 Know-How 指可迁移的实践判断：在什么场景下模型表现好，什么场景下会失败，失败后怎样改 prompt、上下文、工具、评测和产品路径。它像做实验时留下的实验记录，价值同时落在结论和“为什么会这样”的可复现线索上。

### 本节拥有的核心定义

本节只定义 Agent、Know-How、Agent framework、Agent loop 与一线洞察。后文会继续展开上下文、benchmark、RAG、Subagent、Bitter Lesson、工具设计和 memory，本节仅做必要的方向性铺垫。

## 1.2 Agent：一个由模型驱动的循环

讲者对 Agent 的定义非常朴素：Agent 的核心是一个循环。模型读取当前状态，判断下一步是调用工具、执行动作还是直接回答；得到新状态后，再进入下一轮判断。这个过程可以用一句伪代码概括：

```
1 while not done:
2     state = observe()
3     decision = model(context=state)
4     result = act(decision)
5     update_context(result)
```

Listing 1: Agent loop 的最小形态

这段循环背后有几个关键动作。第一，`observe()` 表示模型看到的状态，包括用户消息、系统指令、工具结果、文件内容、错误日志等。第二，`model(context=state)` 表示模型在上下文里做决策。第三，`act(decision)` 可以是工具调用、写文件、搜索、执行命令，也可以是给用户回复。第四，`update_context(result)` 表示把执行结果带回下一轮。

讲者在 00:02:59–00:03:30 强调，写出这个基础 Agent loop 的成本已经很低。在 web coding 变成熟以后，重新实现一个类似 Kimi Code CLI 或 Claude Code 的最小循环，甚至可能只需要少量 prompt 与工程胶水。这个判断听起来刺耳，却是本节的第一根钉子：如果最小循环本身容易复刻，真正的壁垒就不会长期停留在“我也有一个 Agent 循环”上。

## 1.3 框架抽象的价值为什么会变薄

讲者回顾 Prompt Flow、LangChain、LangGraph 这类框架时，给出的判断很直接：它们想在应用与大模型之间建立一层抽象。早期模型 API 混乱、工程范式未稳定，这层抽象可以帮助开发者快速

做 demo、组织调用链、封装工具和状态。但当 OpenAI 与 Anthropic 风格的 API 逐渐被各家兼容以后，“换模型”的接口成本下降了，框架作为兼容层的长期价值也随之降低。

这里要区分两个层次。接口兼容层解决的是“代码能不能接上”的问题，例如消息格式、工具 schema、流式输出和错误处理。行为兼容解决的是“同一套 prompt 与工具设计能不能继续得到同样结果”的问题。讲者真正批评的是前者被高估了，因为后者才是 Agent 产品里昂贵的部分。

举例说，一个团队把 Agent 从模型 A 切到模型 B，HTTP 请求也许只改几行，甚至一行不改。但模型 B 对系统提示词的遵循方式、工具选择的激进程度、长上下文里的注意力分配、对错误日志的修复策略，都可能与模型 A 不同。于是团队要重新调 prompt、重新设计工具描述、重新跑评测、重新观察失败案例。接口像插头，行为像电器本身；插头兼容不代表电器工作方式一致。

### 框架抽象的真实边界

Agent framework 可以降低入门门槛，也适合快速原型。但长期壁垒通常来自行为层的经验：模型如何读上下文、何时乱调用工具、何时需要额外约束、哪些任务应该交给检索或脚本。框架把调用包装得更顺手，一线 Know-How 才决定产品能不能稳定完成任务。

## 1.4 真正的壁垒：在一线发现模型边界

从 00:03:30 开始，讲者把重心转到“模型边界”。大模型对所有人都是新东西，即便是模型提供商，也不天然掌握每个垂直场景里的全部失败模式。可积累的高价值经验来自真实使用：和 LLM 反复交互，做 Agent，烧 token，烧算力，观察模型在哪些场景下可靠，在哪些场景下变得脆弱。

这类经验可以称为一线洞察。它有三个特征。第一，具体，通常绑定任务类型、上下文形态、工具设计与用户工作流。第二，可操作，能转化为 prompt 改法、上下文构造策略、评测样例或产品限制。第三，更新快，因为模型版本、价格、上下文长度和工具协议都在变，去年有效的方法今年可能只剩一半效果。

讲者用 Manus 与模型提供商自带 Agent 的对比来说明这一点。Manus 作为应用方，并不拥有最底层模型，却能在不少任务上做出比通用 Agent 更好的完成效果。原因不神秘：应用方把一线收集到的模型理解、任务拆解、工具组织和释放能力的方法落到了产品里。模型能力是一块矿，一线产品经验决定开采方式。

## 1.5 发现问题本身就是高价值资产

本节后半段最值得记住的一句话是：在 AI 时代，发现问题的价值可能比直接给出答案还高。讲者在 00:05:04-00:05:51 解释了原因：做 Agent 时不可能测完所有场景。很多失败属于“你不知道自己不知道”的问题，也就是 unknown unknowns。团队连某个任务会失败都不知道，就无法设计修复方案。

一旦问题被发现，情况就变了。用户反馈、线上日志、评测回放、工具调用轨迹会把失败从黑暗里拖出来。它可能是上下文污染，可能是工具描述不清，可能是缓存被动态 prompt 打碎，可能是模型在长上下文里遗漏关键信息。问题被命名以后，解决路径通常会出现：补评测、改 prompt、改工具、改上下文保留策略，或者干脆限制产品承诺。

这个逻辑可以类比软件工程里的 bug report。一个隐藏 bug 的危险不在于修起来一定困难，而在于没人知道它存在。Agent 工程更夸张，因为模型行为有概率性，失败模式会被任务、上下文长度、工具返回、模型版本共同触发。发现一个稳定复现的失败场景，等于为团队增加了一条评测基线，也等于给后续产品决策增加了一盏灯。

## 不要把“会调用模型”误判为“懂 Agent”

会接 API、会包一层框架、会写一个循环，只说明系统能跑起来。懂 Agent 需要知道它怎样失败：失败发生在输入、上下文、工具、模型选择、长任务规划、成本约束，还是用户目标本身不清晰。这个判断来自一线样本，无法只靠读框架文档获得。

### 1.6 从 S1 走向 S2：第一条具体 Know-How 是上下文

S1 建立了整场分享的判断：Agent 的真实壁垒来自一线 Know-How，尤其是对模型行为边界的持续再判断。接下来问题自然变成：这些 Know-How 里最底层、最经常被忽略的一条是什么？

讲者给出的第一个答案是上下文。因为 Agent 的每一轮决策都发生在上下文里，工具、指令、历史、文件、错误、图像、音频，最后都要以某种形式进入模型。上下文构造得好，模型像一个知道案情的工程师；上下文构造得乱，模型像在一堆未分类材料里临时猜答案。S2 会把这条线展开到 OpenAI Harmony、prompt cache、cache hit/miss 和 prefill 延迟。

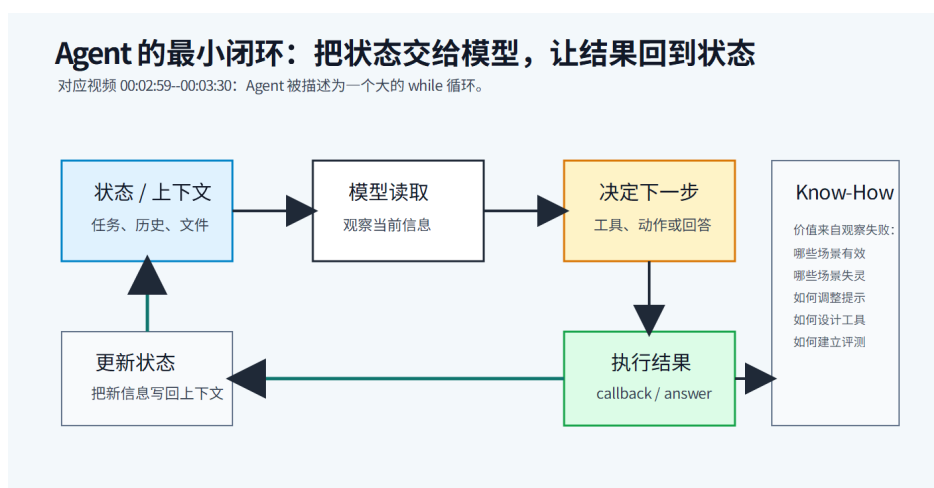


图 1: Agent 的最小闭环：模型读取当前状态，决定下一步行动，工具回传或回答再写回上下文；这个循环越长，越依赖团队对失败场景的 Know-How。

### 1.7 本章小结

本章的重点可以压缩成四句话。第一，Agent 的最小形态是“观察状态、选择行动、执行、再循环”的 Agent loop，基础实现成本已经很低。第二，Agent framework 的兼容层价值会随着 API 收敛而变薄，真正昂贵的是模型行为切换后的重新调校。第三，Know-How 是从真实任务、失败案例和垂直场景里总结出的实践判断。第四，发现问题本身就是资产，因为它把 unknown unknowns 变成可复现、可评测、可修复的工程对象。

## 2 上下文是一切：从 Harmony 到缓存命中

### 2.1 为什么上下文比代码更接近 Agent 的核心

承接 S1 的判断，讲者在 00:06:35–00:07:00 把第一条具体 Know-How 指向上下文。他甚至说，在可见的未来，上下文依然是一切的核心。理由很直接：代码越来越多由 AI 生成，Agent 产品的差异逐渐转移到“你准备让模型看到什么，以及以什么顺序、什么格式、什么稳定性让模型看到”。



## Agent 框架本身价值较低

- 类 langchain/langgraph 框架，希望在 llm 和应用之间构建一层抽象，但提供的价值较低
- Agent 本身就是一个大的 while 循环，研发成本较低，在 vibe coding 成熟下，成本更低。模型看状态 → 决定下一步 → 执行工具或回答 → 循环
- LLM 对全世界来说都是新的，OpenAI 和 Anthropic 并不掌握更多

图 2: 演讲者把 Agent 框架的价值拆成三层：框架抽象层的迁移收益有限，Agent 主体近似 while 循环，真正稀缺的是一线试错得到的模型行为经验。

这里的上下文，指模型一次调用时可以读取的全部输入材料。它包括 system prompt、developer prompt、user prompt、历史对话、工具定义、工具返回、文件片段、图片、音频转写、环境信息和中间执行轨迹。用更工程化的话说，上下文就是模型在决策前拥有的“工作台”。工作台上摆放的材料越贴合任务，模型越容易做对；材料混乱、冗余、变化无常，模型就会在成本、延迟和正确性上一起付账。

### 上下文的工作定义

在本章里，上下文指一次模型调用进入模型侧的全部输入表示。它既包括用户问题和 prompt 文本，也包括工具 schema、工具调用结果、历史消息、多模态输入和动态环境信息；这些材料都会成为模型判断的一部分。

## 2.2 OpenAI Harmony：所谓“魔法”会被序列化成本

讲者在 00:07:11–00:08:24 用 OpenAI Harmony 举例，说明许多看起来像 API 魔法的能力，最后都会被编码成模型可读的上下文。左边是开发者熟悉的结构化概念：system prompt、developer prompt、user prompt、tool call、工具名称、description、parameters、JSON Schema。右边则是 Harmony 风格的带标签序列，把这些结构化内容变成模型输入里的片段。

这个例子想破除一个误解：tool call、JSON mode、工具参数约束看起来像“模型之外的功能”，但它们和模型交互时仍要通过某种表示进入上下文。工具的 name、description、parameters 并不会凭空影响模型；它们会变成模型看到的文本或标记结构。模型根据这些信息决定是否调用工具、怎样填参数、怎样解释工具返回。

讲者也谨慎提到，他不确定开源的 OpenAI Harmony 材料是否已经覆盖多模态输入。但对本章结论影响不大：图片、语音等输入进入模型时，同样需要被编码成模型能够处理的表示。对工程师而言，关键在于先承认所有输入最终都会占用上下文预算、影响模型注意力，并参与缓存命中策略；某个实现细节是否公开，属于第二层问题。

## 2.3 上下文构造：把什么放进去，比调用什么 API 更要命

如果一切都会变成上下文，那么 Agent 的核心问题就变成“上下文构造”。这里的构造至少包含四个选择。

第一，内容选择：哪些信息应该进入这一轮模型调用。比如用户目标、当前文件、错误日志、最近工具结果通常很有价值；无关历史、过期目录列表、冗余文档可能只会增加噪声。第二，顺序选择：稳定、全局、长期有效的指令应放在更靠前的位置，临时观察和工具结果放在更靠近当前任务的位置。第三，粒度选择：给完整文件、片段、摘要，还是只给路径和检索工具。第四，稳定性选择：哪些内容必须保持跨调用一致，哪些内容可以动态追加。

这四个选择互相牵连。给得太少，模型缺证据；给得太多，模型被噪声拖慢；给得太动态，prompt cache 失效；给得太稳定，又可能把临时信息固化到系统层，造成污染。上下文工程的难点就在这里：它同时是正确性工程、成本工程和产品体验工程。

### 上下文构造的第一原则

不要把“能塞进上下文”当成“应该塞进上下文”。上下文窗口像会议桌，能坐更多人不代表讨论会更清楚。每一段材料都应该回答一个问题：它帮助模型做出当前决策吗？它会破坏缓存吗？它会在后续轮次里变成误导吗？

## 2.4 Prompt cache：大规模 Agent 的成本纪律

从 00:08:51 开始，讲者转向 prompt cache。Prompt cache 指模型服务侧复用稳定前缀上下文的机制。当多次请求拥有相同的前缀，服务商可以复用已经处理过的 prompt tokens，从而降低成本与 prefill 延迟。cache hit 表示缓存命中，cache miss 表示上下文发生变化导致无法复用。

先用中文直观解释：一次 Agent 对话往往有很多轮。若每一轮的前半部分 system prompt、工具说明、固定规则都保持一致，模型服务可以把这部分当作“已经读过的教材”。下一轮只需要处理新增内容，成本和延迟都更低。若团队把当前时间、文件列表、目录列表、技能列表等频繁变化的信息塞进前缀，每一轮教材都变了，缓存就会失效。

这个成本直觉可以写成一个轻量公式：

$$E[C] = p_{\text{hit}} C_{\text{hit}} + (1 - p_{\text{hit}}) C_{\text{miss}}$$

其中， $E[C]$  表示一次调用的期望成本或期望延迟； $p_{\text{hit}}$  表示缓存命中概率； $C_{\text{hit}}$  表示缓存命中时的成本或延迟； $C_{\text{miss}}$  表示缓存未命中时的成本或延迟。讲者给出的工程估计很直接：当 cache miss 发生时，价格可能出现十倍甚至十几倍增长，因此通常有  $C_{\text{miss}} \gg C_{\text{hit}}$ 。

这个公式的教学意义在于，它把“缓存命中率”变成产品级变量。团队若把  $p_{\text{hit}}$  从 0.9 打到 0.3，即使单次模型质量完全一样，整体成本和延迟也会明显恶化。Agent 产品的多轮特性会放大这个差异，因为一次用户任务可能触发十几次甚至几十次模型调用。

## 2.5 Prefill：为什么缓存命中也会影响延迟

Prefill 是模型生成答案之前处理输入 token 的阶段。用户常感知到的首字延迟，很多时候就包含 prefill 时间。上下文越长，模型越需要先“读题”；缓存命中时，稳定前缀已经被复用，模型只需处理新增部分，因此首字延迟会下降。

这也是 Agent 里 cache hit 的第二层价值。它同时省钱并让交互更顺。命令行 Agent、编码 Agent、长任务 Agent 都需要频繁往返。如果每一步都因为动态系统提示导致 full cache miss，用户看到的就



会是卡顿、等待和成本飙升。对产品来说，延迟属于会改写用户信任的核心体验变量，它会影响用户是否愿意把长程任务交给 Agent。

## 2.6 哪些动态信息最容易打碎缓存

讲者在 00:09:31–00:10:13 特别点名了一类常见错误：把当前时间、环境信息、文件列表、skill list、directory list 直接放进 system prompt。它们看起来“有帮助”，实际会让几乎每次请求的前缀都变化，从而造成 cache miss。

这些信息的问题不在于没有价值，而在于放错位置。当前时间可能有用，但它应当进入更靠后的动态消息，或只在任务确实依赖时间时提供。文件列表和目录列表也可能有用，但更适合按需检索、工具返回或局部上下文，而不适合每轮都写进稳定 system prompt。Skill list 若允许用户增删，更会让前缀频繁变化；这类内容需要做分层、延迟加载或工具化暴露。

### 缓存敏感内容清单

时间、随机 ID、环境变量、完整文件树、可变 skill list、可变工具列表、用户临时配置，都应谨慎进入稳定前缀。它们一旦放在 system prompt 的早期位置，就可能让后续多轮 Agent 调用全部 cache miss。

讲者还提到 Anthropic 曾有过让 Claude Code 用户共享某个 system prompt 前缀的优化思路，用来提高第一轮回答时的效果和缓存收益；他同时指出这种做法后来已经变化。这里不需要把它当成固定产品事实，值得学习的是背后的工程方向：大规模 Agent 产品会把 prompt 稳定性当成基础设施问题，而非只当 prompt 文案问题。

## 2.7 从 S2 走向 S3：缓存纪律之后是评测纪律

S2 讨论的是成本与延迟压力：上下文怎么构造，会直接影响 cache hit、prefill 和整体使用成本。下一步自然会进入正确性压力：即使上下文和缓存都设计得很好，换模型、换版本、换价格档位之后，Agent 还能保持同样行为吗？

这就引出 S3 的 benchmark 与 baseline。缓存解决的是“同样的上下文如何更便宜、更快地被处理”；评测解决的是“不同模型或不同 prompt 处理后，结果是否仍然符合产品目标”。这两条线合在一起，才是 Agent 工程里真正能落地的纪律。

## 2.8 本章小结

本章围绕一个判断展开：上下文是一切。OpenAI Harmony 的例子说明，system prompt、developer prompt、user prompt、tool call、JSON Schema 和工具描述都会被转成模型输入里的表示；所谓 API 魔法，最终仍要进入上下文。Prompt cache 则提醒团队：稳定上下文前缀能降低成本和 prefill 延迟，动态信息放错位置会造成 cache miss。用公式看，期望成本由  $p_{hit}$ 、 $C_{hit}$  与  $C_{miss}$  共同决定；当  $C_{miss}$  远大于  $C_{hit}$  时，缓存命中率就是产品经济性的核心变量。

# 3 模型切换与评测基线：别相信官方跑分

上一章已经把问题推进到上下文工程：同样一次 Agent 调用，system prompt、developer prompt、工具描述、工具回传都会进入模型上下文，并且会影响 prompt cache 的成本和延迟。本章继续沿着

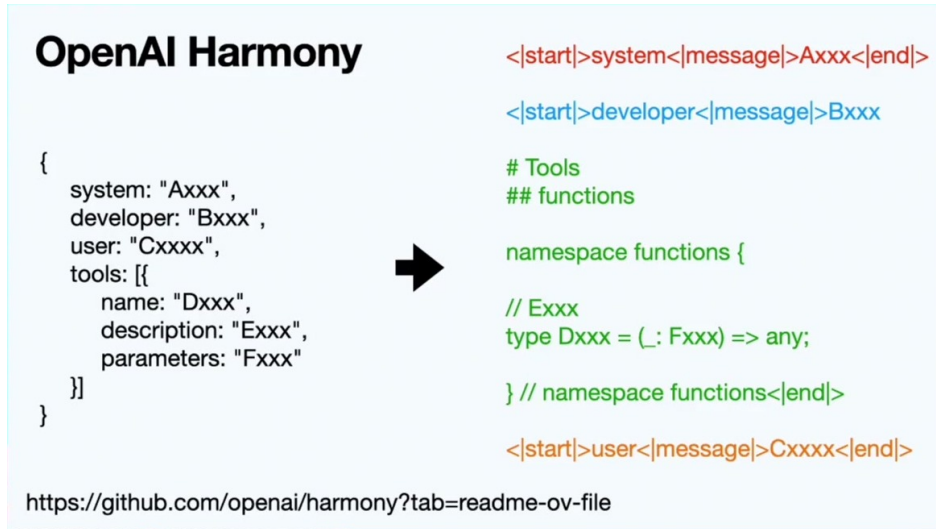


图 3: OpenAI Harmony 示例展示了结构化 API 输入如何被序列化为带标签的文本：system、developer、user、tools 最终共同进入模型上下文。

## Context

- Context 在可见的未来依旧是一切的核心，“你准备如何构造你的上下文”
- 一切都是上下文, 包括 函数、图片、声音 等等一切你的 api 输入
- Cache 命中, 降低 token 成本、减少延迟
  - system prompt 要稳定, 不要把 时间、环境等可能随时改变的动态信息放入

图 4: 上下文工程的成本约束：cache hit 能降低 token 成本和延迟，因此 system prompt 应尽量稳定，时间、环境等易变信息要谨慎放置。

“工程约束”往下走：即使上下文放得很稳，团队仍然会遇到另一个硬问题：模型版本在变，供应商在变，价格在变，官方榜单也在变。产品该怎样判断“该不该换模型”？

讲者在 00:10:35–00:11:26 给出的判断很直接：不同 SOTA 模型的 instruction following 能力、擅长领域、对同一套 system prompt 的服从方式，都不会完全一样。广义任务上看起来接近的两个模型，落到一个已经调过 prompt、调过工具、调过异常处理的 Agent 产品里，差异会被放大。这里的工程麻烦在于，LangChain 或 LangGraph 这类框架可以帮助团队快速搭起 demo，让“应用逻辑不动、底层模型可替换”看起来成立；可一旦产品进入反复调优阶段，模型切换就会带来真实迁移成本。

### SOTA 不等于同质化

SOTA 是 state-of-the-art 的缩写，表示某个时期在公开任务或综合能力上处于前沿的模型。它只说明“总体能力很强”，不能推出“在你的产品里行为一致”。在 Agent 应用中，模型能力至少要拆成三层看：通用语言与推理能力、领域任务能力、对既有 prompt 和工具协议的服从能力。第三层最容易被官方 leaderboard 掩盖。

## 3.1 模型切换的成本在哪里

讲者对模型切换成本的说法很有一线味道：它“比你想象的要高”，同时又“比你想象高的要低”。这句话听起来绕，实际是在拆两种极端误判。

第一种误判是把框架抽象当成迁移完成。框架层面把接口统一成‘chat.completions’、tool call、message list，确实能让调用代码少改很多；可 Agent 的行为还受模型习惯影响。一个模型对“先计划再执行”的理解、对工具失败的容忍、对长 system prompt 的注意力分配、对输出格式的固执程度，都会影响任务结果。接口兼容像电源插头规格相同，插上能通电；行为兼容像两台机器在同一条产线上能否打出同样精度的零件，后者更难。

第二种误判是把模型切换想成不可承受。讲者没有建议永远锁死某个模型。他真正强调的是：如果已经确定了产品领域，就不要“乱换”；如果要换，需要把切换变成评测问题。也就是说，迁移成本可以被工程化管理，前提是团队手里有自己的尺子。

### 模型切换的判断原则

当一个 Agent 已经经过多轮 prompt、工具描述、错误处理和上下文结构调优后，模型切换应当被视为一次产品行为回归测试。官方榜单只能提示候选模型，不能替代应用自己的验收集。

## 3.2 先搭 demo，再建立应用 Benchmark

00:11:43–00:12:00 这一段有一个容易被误读的转折。讲者前面提醒框架抽象本身难以成为长期壁垒，随后又说做 demo 时希望大家先用 LangChain。这里的重点在阶段差异：demo 阶段要快，生产阶段要稳。LangChain 的价值在于把一套架子搭起来，让团队尽早看到真实任务链路；一旦链路跑通，就要尽快建立自己的 Benchmark。

Benchmark 可以翻译成评测基准或基准测试。在这里，它指一组能代表产品真实场景的任务集合，包含输入、期望行为、评分方法和重复运行策略。baseline 是在某个固定模型、固定 Agent 配置、固定评测集上得到的基线分数。没有 baseline，团队讨论“新模型更好”“便宜模型够用”“prompt 改动有收益”都会变成感受。

如果团队一开始缺少自有样例，也可以先借助相近领域的开源 Benchmark 冷启动。它们不能替代产品验收集，却能先暴露输出不稳定、工具调用失败、格式不合规等问题；等真实用户任务积累起来，再逐步沉淀成自己的 Benchmark。

一个可落地的流程可以写成四步。

1. 先用框架或最小 Agent loop 搭出 demo，让真实用户任务流动起来。
2. 从 demo 和早期用户反馈里抽出代表性样例，覆盖正常路径、边界条件、失败恢复和高价值场景。
3. 把当前模型与当前 Agent 配置放进 Benchmark，重复运行至少 5-10 遍，计算平均分和波动。
4. 把这个结果冻结为 baseline，之后每次换模型、换 prompt、改工具协议，都和它比较。

讲者特别强调“不要只跑一遍”。大模型输出有随机性，即便 temperature 很低，也可能因为采样、服务端实现、工具回传细节而出现差异。只跑一次相当于拿一枚硬币抛一次就判断硬币偏不偏；工程上需要看均值，也要看方差。

#### Benchmark 与 baseline 的本章定义

Benchmark 是产品自己的任务测量仪，baseline 是在该测量仪上跑出来的参考刻度。前者回答“用什么测”，后者回答“当前水平是多少”。如果没有这两个东西，模型切换、降本替换、prompt 改版都会缺少可复现的判断依据。

### 3.3 用轻量符号描述应用基线

为了把这件事说清楚，可以引入一个轻量符号。设某个 Agent 配置在应用 Benchmark 上的分数为：

$$B_{\text{app}}(m, p, t) = \frac{1}{n} \sum_{i=1}^n s_i(m, p, t)$$

其中：

- $B_{\text{app}}$ ：application-specific benchmark score，即应用自己的基准分数；
- $m$ ：被评测的模型版本，例如某个供应商的新模型或更便宜的小模型；
- $p$ ：当前 prompt、工具描述、上下文结构等 Agent 配置；
- $t$ ：真实任务集合，也就是 Benchmark 中的测试样例；
- $s_i$ ：第  $i$  次重复运行得到的评分；
- $n$ ：重复运行次数，讲者建议至少 5-10 次来抵消模型 variance。

这个公式没有复杂数学，作用只是逼团队把“感觉更聪明”变成“在我们的任务上、在这套配置下、重复运行后的平均表现”。variance 可以用下面的直觉理解：如果某模型平均分很高，但每次运行忽高忽低，它在自动化 Agent 里依然可能危险，因为用户遇到的是单次结果，线上系统承受的是尾部失败。

#### 官方跑分的盲区

官方 leaderboard 往往衡量公开数据集上的综合能力，产品 Benchmark 衡量的是“你的用户到底会不会成功”。两者相关，可替代关系很弱。尤其在 Agent 场景中，工具调用、上下文长度、错误恢复、输出协议稳定性，常常比单题推理分数更决定产品体验。

### 3.4 新模型、降本模型，都要过同一把尺

00:12:34–00:13:11 的建议非常具体：现在模型更新很快，可能一个月就有小版本；新模型出来后，不要先看官方跑分，要拿它在自己的 Benchmark 上跑一遍。若业务有降本增效需求，也可以把更便宜的模型放进同一套 Benchmark：如果得到接近的效果，就有切换到低价模型的依据。

这里需要补一层产品经济学。Agent 的模型成本同时来自单次 token 价格、失败重试、工具调用次数、人工接管率、延迟导致的用户流失。一个便宜模型如果让任务多跑三轮，实际成本可能更高；一个贵模型如果显著减少失败恢复和人工介入，单位成功任务成本可能更低。所以 Benchmark 最好不要只给“正确/错误”二值分，还要记录运行步数、工具调用数、耗时、失败类型。

可以把一次评测结果拆成一个小表：

| 指标    | 含义               | 对产品的影响   | 适合记录方式            |
|-------|------------------|----------|-------------------|
| 成功率   | 任务是否达成           | 直接影响用户结果 | 自动评分或人工复核         |
| 平均步数  | Agent 走了多少轮      | 影响成本与延迟  | 日志统计              |
| 工具失败率 | tool call 是否频繁出错 | 影响稳定性    | 错误码与回传分类          |
| 输出合规率 | 格式、字段、约束是否满足     | 影响下游系统接入 | JSON Schema 或规则校验 |

这张表体现的是本章的核心语气：别把 Benchmark 做成论文比赛，要把它做成产品驾驶仪。驾驶仪不需要解释宇宙，它需要告诉工程师现在能不能起飞、哪里在抖、油耗是否失控。

### 3.5 长上下文退化：越塞越多不等于越懂

从 00:13:11 开始，讲者把 Benchmark 话题转到长上下文。随着  $L_{\text{ctx}}$  变长，模型能力会下降，而且下降可能很快。这里的  $L_{\text{ctx}}$  表示上下文长度，包含系统提示、工具说明、历史消息、文件内容、检索结果、工具回传等所有进入模型的 token。

直觉上，很多人会以为“把更多材料塞进去，模型就知道更多”。问题在于模型的注意力资源有限，且分布很难绝对均匀。长上下文像把一整本书、会议纪要、错误日志、用户要求一起摊在桌上，让一个人立刻回答细节题和综合推理题。材料越多，寻找关键证据越难；同时上下文本身还会增加成本。

讲者举了一个很形象的测试：把整本英文版《哈利波特与魔法石》放进上下文，然后用中文提问。一类问题是回忆型细节，例如哈利最开始去了哪个巷子；另一类问题要求基于小说开头和结尾的信息做推理。前者测试 recall，后者测试 reasoning。一个模型可能能找到事实细节，却在跨越长距离信息的综合推理上变差；也可能在某些位置的信息上注意力不足，导致答案看似流畅却漏掉关键线索。

长上下文退化可以用一个简化函数表达：

$$Q(L_{\text{ctx}}) = Q_0 - d_{\text{recall}}(L_{\text{ctx}}) - d_{\text{reason}}(L_{\text{ctx}})$$

其中：

- $Q(L_{\text{ctx}})$ ：上下文长度为  $L_{\text{ctx}}$  时的任务质量；
- $Q_0$ ：短上下文或理想上下文下的参考质量；
- $d_{\text{recall}}$ ：由于注意力稀释导致的检索、定位、回忆损失；
- $d_{\text{reason}}$ ：由于跨段整合困难导致的推理损失。

这个表达式只承担提醒功能：读者需要把“找得到”和“想得通”分开测。很多 Agent 失败发生在第二类：它拿到了足够材料，却把材料之间的关系推错了。



## 长上下文的双重账单

上下文越长，产品同时支付两笔账：一笔是显性的 token 成本和延迟，另一笔是隐性的能力损失。前者能在账单里看到，后者常常表现为偶发错误、漏看约束、推理断裂和用户觉得“它明明读过，为什么还犯错”。

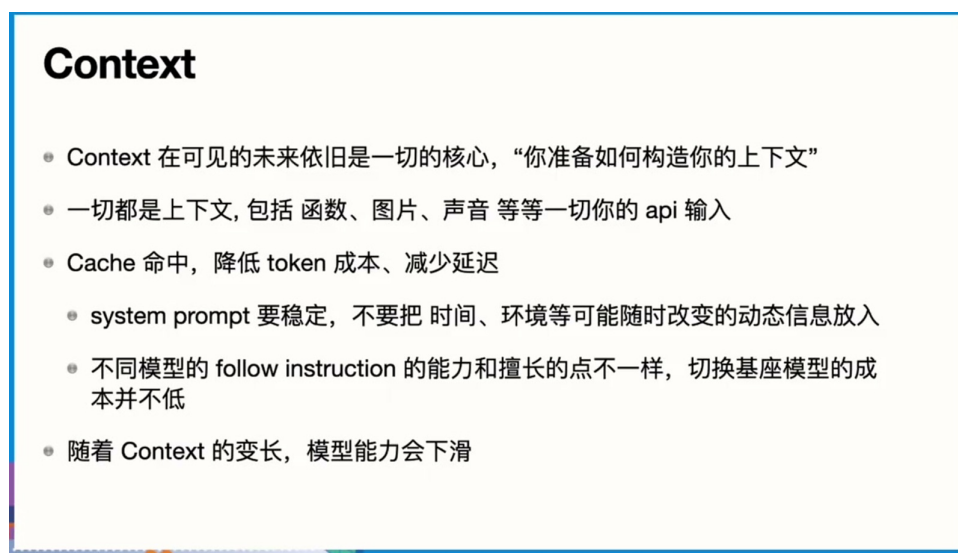
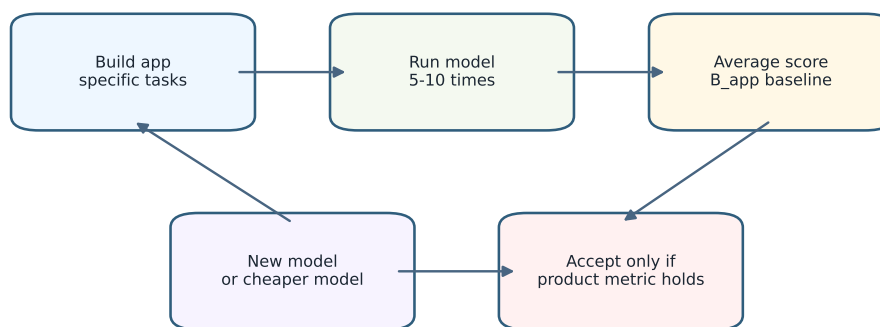


图 5: 讲者在同一张 Context slide 上把模型切换和长上下文串到一起: 不同模型的 instruction following 能力、擅长领域和 system prompt 响应会变, Context 变长后模型能力也会下滑。

### Benchmark discipline for model switching



Official leaderboard is a reference; product-specific variance decides the release.

图 6: 应用级 benchmark 的闭环: 先构造贴近产品任务的测试集, 再用 5-10 次重复运行估计平均表现, 形成  $B_{app}$  baseline; 新模型、廉价模型和 prompt 调整都应回到这条基线复测。

## 3.6 本章小结

本章把“模型选择”从品牌和榜单拉回工程现场。讲者的实践建议可以压缩成三句话。

- SOTA 模型在大面上可能接近, 落到具体 Agent 产品里会因为 instruction following、领域能力和 prompt 适配产生差异。

- 做应用时可以借助 LangChain 快速搭 demo，但 demo 跑通后要尽早建立应用 Benchmark，并用重复运行得到 baseline。
- 新模型、便宜模型、prompt 改版、工具协议改版，都应该回到同一套  $B_{app}$  上比较；长上下文还要分开测试 recall 与 reasoning。

这也自然引出下一章：一旦 Benchmark 告诉团队“长上下文会掉能力”，真正的难题就变成如何构造上下文。哪些搜索过程该保留，哪些错误记录该删，什么时候要动态提示，什么时候压缩会制造幻觉，这些都属于 Agent 一线工程的硬活。

## 4 上下文管理的工程难题：检索、错误、动态提示与压缩

上一章用 Benchmark 和长上下文退化说明了一件事：上下文越长，成本越高，能力也可能下降。本章进入更细的工程现场。讲者在 00:14:30 开始说，模型对整个上下文的理解是非常 tricky 的。这里的 tricky 指输入中的每一段材料都可能改变模型的行为：搜索过程、错误记录、动态提醒、few-shot 示例、压缩摘要，都会成为模型判断下一步的证据。

本章的关键问题是：当上下文空间有限时，Agent 产品到底该保留什么、丢掉什么、什么时候插入提醒、什么时候压缩历史？没有一条规则能放之四海。讲者反复强调，今天讲的许多东西都没有唯一正确答案，团队要结合自己的场景和 Benchmark 做取舍。

### 4.1 RAG 的问题：答案以外，搜索过程也在教模型

00:14:37-00:16:18 这一段以 Cursor 和 Claude Code 的差异为例。Cursor 早期很重视 RAG：打开项目后索引整个代码库，用 embedding 找到相关代码片段，再把片段放进模型上下文，让模型基于这些片段继续写代码。RAG 即 retrieval-augmented generation，通常译为检索增强生成；embedding 是把文本或代码映射到向量空间，方便相似度检索。

这个方法在自然语言文本里很常见，但代码场景有两层疑问。第一，很多 embedding 模型主要从自然文本训练而来，对代码结构、调用关系、约定俗成的项目模式是否仍然有效，需要实测。第二，RAG 很容易把“找到答案的过程”剪掉，只把最终片段喂给模型。讲者认为，这会损失一类关键上下文。

人类开发者进入一个陌生开源项目时，很少只看最终文件。开发者会先看目录，猜模块边界；再用 ‘grep’ 找符号，用 ‘glob’ 找文件模式；途中看到命名风格、测试结构、错误处理习惯、配置入口。这个过程像在城市里走路，最终目的地很重要，沿路建立的方位感同样重要。Claude Code 风格的 ‘grep’ / ‘glob’ 逐层探索，把这些“沿路看到的東西”也留在上下文中，模型因此更像在学习一个项目。

#### 渐进式检索的本章定义

渐进式检索指 Agent 像开发者一样用 ‘grep’、‘glob’、文件读取、目录观察等熟悉工具逐步探索代码库。它的价值同时在于定位答案，并让模型在搜索过程中吸收项目结构、命名习惯、依赖关系和局部约束。

这解释了一个反直觉现象：如果团队设计了一个更“聪明”的检索器，能快速把答案片段送到模型面前，模型未必干得更好。因为被删掉的搜索路径本身也是上下文。讲者的结论很朴素：每一种上下文构造方案都要回到 Benchmark 测试，不能只凭“更省 token”“看起来更快”判断。

## RAG 在代码 Agent 中的盲区

RAG 容易优化“相关片段召回”，却弱化“项目理解形成”。代码任务的信息分布很密，正确修改常常依赖调用链、测试约定、隐含风格和历史错误。只给模型最终片段，可能让它少看了决定行为的上下文路径。

### 4.2 失败工具调用：删掉省 token，留下防复读

第二个难题是错误记录要不要保留。00:16:29–00:17:11 中，讲者描述了 coding Agent 常见的一幕：模型调用某个工具，错了三四次，最后一次调对了。做 context 优化时，团队很自然会想删掉前三条失败记录，只留下正确调用。这样做的好处非常明确：省 token，缩短上下文，减少模型被无关信息干扰。

问题也同样明确：如果失败记录全被删掉，模型可能再次犯同样的错。对人类来说，“这条路刚才走不通”是一条很有价值的负样本；对模型也是如此。错误记录有时就是证据：它记录边界条件、工具约束、权限限制、路径命名和参数格式。

可以把这个取舍写成一个小的决策表：

| 策略       | 收益            | 风险              |
|----------|---------------|-----------------|
| 删除失败调用   | 节省 token，减少噪声 | 模型可能重复相同错误      |
| 完整保留失败调用 | 保存负样本和调试路径    | 上下文膨胀，注意力被稀释    |
| 压缩成错误摘要  | 保留关键教训，成本较低   | 摘要若漏掉细节，会误导后续步骤 |

更稳的做法通常是分层保留：对无意义的拼写错误、重复超时、明显网络抖动，可以压缩；对参数格式、文件路径、权限边界、测试失败原因等会影响后续决策的错误，要至少保留结构化摘要。这里的“结构化”不需要复杂，哪怕只记录“失败动作、失败原因、下一次避开的规则”，也比简单删除更可靠。

#### 负样本也是上下文

在机器学习里，负样本帮助模型知道边界在哪里；在 Agent 上下文里，失败工具调用也有类似作用。它告诉模型哪些路径刚刚被证伪，哪些参数组合不可用，哪些假设已经被现实打脸。删掉失败记录，会让 Agent 像失忆的调试者一样原地绕圈。

### 4.3 动态提示注入：用规则提醒智能体的悖论

第三个机制是 dynamic prompt injection，动态提示注入。讲者在 00:17:16–00:19:31 讲到，做 coding Agent 或类似 Agent 时，可以根据运行状态动态插入 prompt：当上下文长度接近上限，提醒模型尽快给出答案；当一个 turn 中思考和执行步数过多，提醒模型检查是否遗漏了什么；当处于 plan mode，提醒它此阶段不要写代码。

这个机制确实有用。它把上下文管理从静态系统提示，推进到运行时控制。静态 prompt 像出门前写在纸上的规则；动态注入像导航软件在拐弯前发出的提醒。问题在于，这里有一个听起来有点荒诞的悖论：团队一方面相信大模型很聪明，另一方面又用一套没有智能的业务规则去提醒它。如果改用小模型监控大模型，也会出现类似问题：低能力监督者能否稳定发现高能力模型走偏？

讲者举了 plan mode 的例子。Claude Code 早期 plan mode 的核心约束可以概括为一句话：你现在在 plan mode，不要写任何代码。后来为了更稳定地保证模型在 plan mode 中不修改文件，这类

prompt 可能膨胀到接近上千行，并且系统还会周期性注入强提醒；字幕里的例子是大约每五个 turn 强化一次约束，提醒模型仍处于 plan mode，继续避免写代码。

### 动态提示注入的悖论

动态提示注入能修正模型运行中的偏航，也会带来新成本：prompt 变长、cache 更难稳定、模型更容易 miss out 其他约束。它像给驾驶员加报警器，报警器能救场，报警器太多也会制造噪声。

所以这里的真实问题应改写为“哪些触发条件值得注入”。比较稳的触发条件通常具备三个特点：信号可观测，例如上下文长度、turn 数、工具失败次数；后果严重，例如即将越过上下文上限或即将违反 plan mode；提示足够短，能明确改变下一步行为。反过来，如果触发条件模糊、提示很长、内容重复，就容易把上下文变成一堆警示牌。

## 4.4 Few-shot 与 prompt perturbation：别让模型偷懒复制

第四个点是 few-shot example 和 prompt perturbation。讲者在 00:19:31-00:20:36 描述了一个常见现象：当某个垂直领域的数据处理方式模型总学不会时，团队会给几个示例，引导模型照着做。few-shot 的逻辑很合理：给 A、B、C 三个样例，模型根据样例处理新任务 D。

麻烦在于，如果 D 和 A/B/C 相似但存在关键差异，模型可能变懒，直接复制前三个样例的处理方式。这里的“懒”指模式匹配的副作用：上下文里示例越强，模型越倾向复用示例结构；当新任务需要做不同动作时，强示例会变成错误吸引子。

prompt perturbation 可以理解为提示扰动：仍然注入 few-shot 示例，但让示例内容、顺序、措辞或边界条件发生轻微变化，避免模型把某一个固定模板当成唯一答案。它像训练学生时不能只给三道同构题，否则学生会背题型；要在同一知识点下换数字、换问法、换约束，让学生学规则，而非记答案。

### 提示扰动的核心目的

prompt perturbation 的目标在于削弱 few-shot 示例的过度吸附力。好的扰动让模型看到“同一原则可以适配不同表面形式”，从而降低直接复制 A/B/C 处理流程到 D 的概率。

在 Agent 产品中，这个技巧尤其适合用于格式转换、垂直领域抽取、代码修改模式、数据清洗流程。评测时不要只看平均正确率，还要专门挑“看起来像旧例子、实质上要换动作”的样例，因为这类样例最容易暴露 few-shot 过拟合。

## 4.5 Compaction 与幻觉累积：代码压缩尤其危险

最后一个难点是 compaction，也就是把长会话压缩成摘要，腾出上下文空间。00:20:36-00:21:51 中，讲者说得很重：压缩一定会产生幻觉，尤其在 coding Agent 里，一个 session 一旦 compact，后面常常就很难用了。

原因在于代码的信息密度很高。自然语言里有大量冗余，讲者现场说一大段话，删掉一半，主旨可能还在；代码不一样，一个变量名、一个导入路径、一个布尔条件、一个缩进层级，都可能改变程序行为。压缩代码上下文时，丢失的信息很容易恰好是后续修改需要的关键信息。

更危险的是，模型会尊重当前代码库，也会把前文上下文当成 ground truth。若压缩摘要里已经产生了偏差，后续模型会基于这份偏差继续推理；它可能把上一步摘要中的幻觉当成事实，再写出新的代码或新的解释。于是幻觉在长任务中积累，像账本第一行记错后，后面每一行都越算越偏。



可以把 compaction 的风险链条写成：

原始上下文 → 压缩摘要 → 信息损失 → 错误 ground truth → 后续幻觉累积

其中：

- 原始上下文：包含代码、工具调用、错误记录、用户约束和中间推理的完整历史；
- 压缩摘要：为了节省 token 生成的简化版本；
- 信息损失：摘要漏掉路径、条件、失败原因、边界约束等细节；
- 错误 ground truth：模型把摘要里的偏差当成真实事实；
- 后续幻觉累积：后面的行动继续建立在错误事实上，失败概率随任务长度上升。

### 代码任务里的压缩风险

代码上下文的单位信息量远高于普通口语。压缩摘要如果漏掉一个函数签名、错误码、测试断言或路径约束，后续 Agent 可能仍然表现得很自信，因为它会把摘要当作可信历史。对 coding Agent 来说，compaction 应当被视为高风险恢复机制，而非无损清理按钮。

更务实的策略是把压缩前后的内容纳入 Benchmark：压缩后继续跑同一类任务，比较成功率、重复错误率、工具调用步数和人工接管率。若压缩不可避免，摘要也应该偏向保留结构化事实：已修改文件、未解决错误、失败过的方案、用户硬性约束、测试命令与结果。漂亮的自然语言总结没有那么重要，能帮助下一轮不走错路才重要。

## Context

- Context 在可见的未来依旧是一切的核心，“你准备如何构造你的上下文”
- 一切都是上下文, 包括 函数、图片、声音 等等一切你的 api 输入
- Cache 命中，降低 token 成本、减少延迟
  - system prompt 要稳定，不要把 时间、环境等可能随时改变的动态信息放入
  - 不同模型的 follow instruction 的能力和擅长的点不一样，切换基座模型的成本并不低
- 随着 Context 的变长，模型能力会下滑
- 上下文的内容对整个项目的影响是很 treaky 的，渐进式披露/发现

图 7：这张过渡 slide 将 S3 的长上下文退化引到 S4 的核心问题：上下文内容会影响模型对整个项目的理解，渐进式披露和搜索过程本身都可能成为模型学习项目结构的一部分。

## 4.6 本章小结

本章把“上下文管理”拆成五个一线难题。

- RAG 能快速提供相关片段，但代码 Agent 还需要搜索过程带来的项目理解；‘grep’ 和 ‘glob’ 的价值在于渐进式探索。



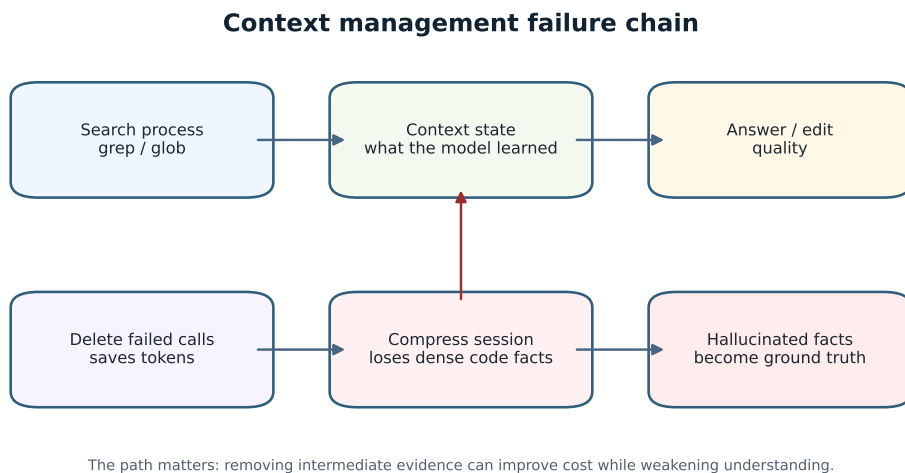


图 8: 上下文管理的失效链条: grep/glob 的搜索过程会让模型逐步建立项目状态; 删除失败调用和压缩会节省 token, 却可能损失代码密集事实, 后续回合再把损失后的内容当成 ground truth。

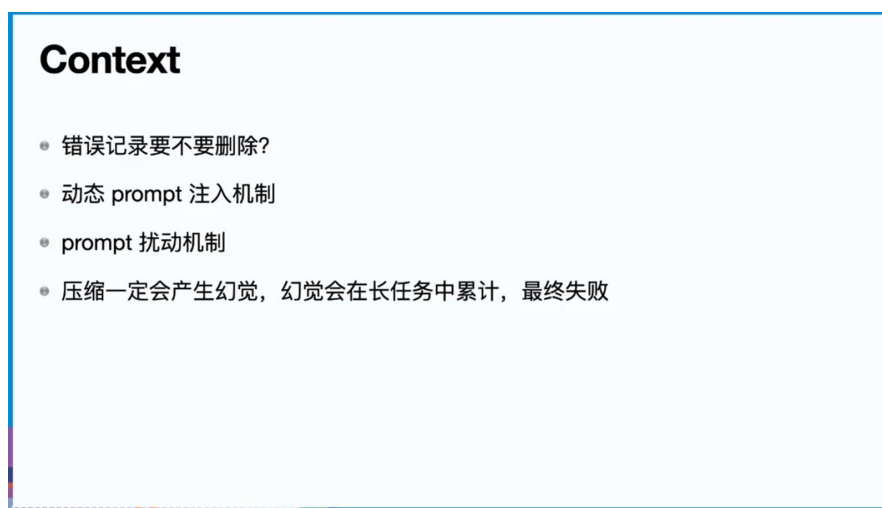


图 9: S4 后半段的四个具体风险: 错误记录是否保留、动态 prompt 注入、prompt 扰动机制, 以及压缩造成幻觉并在长任务中累积。

- 失败工具调用不能简单当作垃圾；它们可能是防止模型复犯错误的负样本。
- dynamic prompt injection 能提醒模型走出长上下文、长步骤或 plan mode 误区，也会增加 prompt 长度和噪声。
- few-shot example 会引导模型，也可能让模型复制旧模板；prompt perturbation 用轻微变化降低过度模仿。
- compaction 在代码任务中会丢失高密度信息，并可能把幻觉沉淀为后续 turn 的 ground truth。

讲者的底层提醒非常朴素：上下文会主动塑造模型下一步怎么想、怎么找、怎么犯错、怎么自我修正。Agent 工程的很多 Know-How，就藏在这些看似琐碎的保留、删除、扰动和压缩选择里。

## 5 多 Agent 协作：Subagent 的优雅与 Agent Team 的风险

前一章谈到压缩和 summary 会在代码任务里制造信息损耗：模型把一段真实搜索过程压成几句话，常常会丢掉路径、失败尝试、隐含约束和文件之间的关系。00:21:51 之后，讲者把这个问题推进到多 Agent 设计：只要 Agent 之间频繁传话，就会遇到同一种损耗，只是损耗发生在“同伴沟通”这个更有吸引力的包装里。

### 5.1 从 router 设计说起：把请求分发给专门角色

讲者先回顾了当时很流行的一类多 Agent 系统：router design。它的基本形态很直观，一个请求先进到 router agent，router 判断任务类型，再把任务转给某个专门 Agent。例如，前端需求交给前端 Agent，后端需求交给后端研发 Agent，设计问题交给设计 Agent。这种架构看起来很像真实公司里的工单分派，也符合工程师对“职责分离”的直觉。

#### router design 的直觉来源

router design 借用了传统软件系统里的路由思想：入口统一、下游专门化、职责边界清楚。它适合描述组织结构，也适合做产品演示，因为用户很容易理解“这个问题由哪个专家处理”。但 Agent 里的 router 多了一层难点：下游 Agent 接收到的通常已经是被改写、筛选或概括过的信息。

这个设计的吸引力在于“像团队”。一个人类团队确实会把任务拆给前端、后端、设计、测试、数据等角色。但人类团队共享大量背景：会议记录、代码仓库、历史争论、口头暗示、团队默契。Agent Team 里每个 Agent 实际拿到的是上下文窗口中的文本。router 一旦把原始任务压缩成“前端问题，请你处理”，下游 Agent 就失去了很多判断依据。

可以把一次 router 分发写成一个简单的信息链条：

$$I_{\text{raw}} \rightarrow S_{\text{router}} \rightarrow A_{\text{specialist}} \rightarrow S_{\text{return}}$$

其中各个符号含义如下：

- $I_{\text{raw}}$ ：用户原始请求、已有代码、工具调用记录等完整输入。
- $S_{\text{router}}$ ：router agent 生成的任务摘要或分派说明。
- $A_{\text{specialist}}$ ：负责某类任务的专门 Agent。

- $S_{\text{return}}$ : 专门 Agent 返回给上层 Agent 的结果摘要。

这条链越长，信息被重写的次数越多。每一次重写都可能改变重点：router 可能漏掉用户真正关心的约束，专门 Agent 可能把自己的假设写成事实，上层 Agent 再把这个事实继续传播下去。

## 5.2 Subagent 模式：隔离任务，减少过程沟通

讲者随后把视角转向 Claude Code 早期的 Subagent 设计。他认为这个实现“非常优雅”，原因并不神秘：它把多 Agent 协作压缩成一次明确委托，而非一串持续聊天。

在这个模式里，主 Agent 通过 task tool 创建一个能力相近的子 Agent。主 Agent 需要在开始时写一个非常详细的 prompt，说明任务、边界、要看的文件、输出要求和判断标准。子 Agent 独立搜索、执行、整理，完成后只返回最终结果。关键约束是：主 Agent 在过程中不能和它来回沟通。

### Subagent 的本质

Subagent 是一次“高质量初始委托”。它用详细任务说明换取过程隔离，用最终结果换取低频沟通。这个模式的重点不在于多了一个 Agent，而在于减少 Agent 之间反复 summary 的次数。

这与上一章的上下文管理形成了直接呼应。代码任务里，探索过程本身有价值；如果主 Agent 与子 Agent 一直聊天，双方就不得不不断概括“我刚才看了什么”“我认为哪里有问题”“下一步可能做什么”。这些概括会把原本丰富的搜索轨迹压成片段。Subagent 设计干脆让子 Agent 自己持有完整过程，最后只交付一个经过整理的结果。

更具体地说，Subagent 模式把沟通成本前置到初始 prompt。主 Agent 要在任务开始时回答几个问题：

- 任务目标是什么，成功标准是什么；
- 哪些文件、目录、日志或测试最相关；
- 哪些修改边界不能越过；
- 输出应当是结论、补丁、测试结果，还是候选方案；
- 不确定时应记录哪些假设，而不能把假设伪装成事实。

这也是为什么讲者强调“最开始的 prompt 要非常详细”。Subagent 的优雅建立在任务说明足够清楚之上。若初始委托含糊，隔离执行就会变成隔离误解；主 Agent 等到最后才发现子 Agent 走错方向，修复成本反而更高。

## 5.3 summary bottleneck: 多 Agent 沟通的真实瓶颈

讲者在 00:23:00 左右给出核心判断：沟通一定意味着 summary，而现在的模型并未被训练得特别擅长这类沟通。这里的 summary bottleneck 可以译作“摘要瓶颈”：一个 Agent 想把自己知道的东西交给另一个 Agent，通常无法原样传递完整上下文，只能抽取重点、压缩过程、删去细节。

### 摘要瓶颈最容易伤害代码任务

代码任务里的“细节”经常就是答案本身：某个失败命令、某个边界条件、某个文件名的拼写、某段调用链里的反常分支。summary 会天然偏向抽象结论，容易把这些低层细节压掉。随后接手的 Agent 只能在不完整地图上继续行动。

这可以用信息量的角度理解。假设原始探索过程包含的信息量为  $H(I_{\text{raw}})$ ，一次摘要后保留下来的有效信息为  $H(S)$ 。在理想情况下，摘要只删去噪声；在真实代码任务中，模型并不知道哪些细节以后会变成关键证据，所以常见关系更接近：

$$H(S) < H(I_{\text{raw}})$$

符号含义如下：

- $H(I_{\text{raw}})$ ：原始任务上下文和探索过程中的有效信息量。
- $H(S)$ ：summary 中被保留下来的有效信息量。
- 不等号表达的是工程直觉，不能当作严格的信息论证明：摘要通常会牺牲未来可能有用的细节。

更棘手的是，summary 的错误很难被下游 Agent 发现。下游 Agent 没看过完整过程，只能相信上游的转述。如果上游说“测试主要失败在认证模块”，下游就会围绕认证模块展开；若真实原因是测试夹具初始化顺序改变，错误方向会被继续放大。

#### 5.4 Agent Team：看起来更像组织，也更容易积累幻觉

讲者并没有把 Agent Team 一棍子打死。他提出了一个谨慎问题：如果某个模型厂商开始做 Agent Team，是否已经针对这种多 Agent 沟通训练过模型，或者引入了新的机制来强化它？这是一个开放判断，不能写成确定结论。

当前实践里的风险仍然很明确。很多 Agent Team 设计会鼓励成员之间互相提醒、互相发送消息、互相接受结论。为了让协作顺畅，系统往往还会提示某个 Agent：如果另一个 Agent 同事提醒了你一件事，你应当把它基本视为正确。这种机制能减少争吵，也会降低纠错能力。

##### 幻觉传播的链式结构

单个 Agent 的幻觉还可能被测试、工具调用或人工检查拦住。Agent Team 中，一条幻觉如果被写进“同事提醒”，再被另一个 Agent 当成可信前提，就会从局部错误升级成团队共识。讲者在 00:23:50-00:24:03 明确指出：幻觉会累积。

这就是 Agent Team 的悖论式风险：它越像一个会沟通的团队，越需要处理“谁的话能信、信到什么程度、如何回到原始证据”这些治理问题。人类团队有代码 review、会议纪要、责任人、测试流水线 and 事后复盘；Agent Team 若只有消息流，很容易把“发过消息”误当作“建立事实”。

#### 5.5 实践判断：什么时候用 Subagent，什么时候警惕 Agent Team

从这一段讲者的判断可以抽出一条实用规则：当任务可以被清楚隔离，并且结果可以被主 Agent 或人类用测试、diff、日志、文件证据验证时，Subagent 很合适。比如让一个子 Agent 搜索某个 bug 的来源、整理一组 API 的调用点、检查某个模块的潜在回归风险。

当任务需要连续协商、跨 Agent 共享大量未压缩上下文，或者要求多个 Agent 互相信任对方的中间判断时，Agent Team 的风险会明显升高。尤其在代码、运维、安全和数据分析场景里，中间事实必须能回到证据，而不能只停留在“某个 Agent 说过”。

## 多 Agent 设计的落地原则

优先选择能减少沟通轮次的结构：一次清晰委托、独立执行、证据化返回、上层验证。只有当模型、工具和训练机制确实能处理跨 Agent 信任时，再扩大到高频消息式 Agent Team。

这也解释了为什么本讲的多 Agent 讨论首先属于上下文工程问题，组织学想象只能解释其中一层。Agent 之间的每一次通信都会进入上下文，每一次 summary 都会改变信息分布，每一次信任转移都可能让幻觉更难被发现。

### Context

- 错误记录要不要删除？
- 动态 prompt 注入机制
- prompt 扰动机制
- 压缩一定会产生幻觉，幻觉会在长任务中累计，最终失败
- Agent team VS Sub Agent

图 10: S5 的承接页把上下文管理中的压缩幻觉，推进到 Agent Team 与 Subagent 的协作取舍：多 Agent 通信一旦依赖摘要传递，就可能把错误判断带入后续任务。

## 5.6 本章小结

本章围绕 00:21:51–00:24:04 的多 Agent 讨论，核心结论可以压缩成三点。

- router design 容易理解，也容易演示，但它会把原始任务压缩成分派消息，下游 Agent 可能失去关键细节。
- Claude Code 早期的 Subagent/task tool 模式优雅之处在于减少过程沟通：主 Agent 给出详细初始 prompt，子 Agent 独立完成任务并返回结果。
- Agent Team 可能随着模型厂商训练而变强，但当前高频消息式协作会遇到 summary bottleneck 和 hallucination propagation，尤其在代码场景里风险很实在。

这一章的过渡意义很强：多 Agent 架构的风险把问题带到了下一章。模型能力不断增强时，Agent 架构究竟应该更复杂，还是更简单？讲者接下来用 Bitter Lesson 给出判断框架。

## 6 Bitter Lesson for Agents: 模型变强时，架构要变简单

00:24:04 之后，讲者引入 Bitter Lesson。这个概念来自人工智能研究中的长期观察：人类精心设计的复杂机制常常能在某个阶段带来收益，但随着算力和模型能力上升，最终更能吃到红利的往往是简单、通用、能充分利用规模增长的方法。放到 Agent 工程里，它给出的提醒很直接：架构应当跟着 base model capability 变化，不能把上一代模型的补丁永久写进下一代系统。



## 6.1 Bitter Lesson: 复杂技巧的有效期

Bitter Lesson 可以译为“苦涩教训”。它苦涩的地方在于，它并不否认人的技巧有效；相反，很多技巧在短期内确实有效。讲者在 00:24:33–00:24:43 提到，当算力或模型能力卡在某个节点时，研究者会用各种优化和复杂机制让模型效果变好，这些机制往往真的能工作。

问题出在时间尺度。一旦下一代能力涌现，上一代技巧可能变成限制。一个为了弥补缺弱模型而设计的固定流程，可能会挡住强模型自己搜索、推理、试错和修正的空间。架构越依赖硬编码规则，越可能把模型框在旧能力假设里。

### Bitter Lesson 在 Agent 里的含义

Agent 架构的先进性不能用复杂度衡量。更可靠的方向是：在当前模型能力下保留必要脚手架，同时持续检查哪些脚手架已经开始拖累模型。模型变强之后，系统要敢于把一部分人类预设机制退场。

可以用一个简化表达来描述架构收益：

$$Q_{\text{agent}} = f(M, A, T, C)$$

符号含义如下：

- $Q_{\text{agent}}$ ：Agent 的实际任务质量，包括正确性、速度、成本和可恢复性。
- $M$ ：base model capability，即基础模型能力。
- $A$ ：Agent architecture，即系统如何组织循环、工具、上下文和角色。
- $T$ ：tools，即工具集合及其描述、参数和返回内容。
- $C$ ：context construction，即上下文构造方式。

这个公式的意义不在于精确计算，而在于提醒： $A$  的最优形态依赖  $M$ 。当  $M$  变强，旧的  $A$  未必仍然最优。继续堆复杂流程，可能让  $Q_{\text{agent}}$  下降。

## 6.2 base model capability 决定 Agent architecture

讲者在 00:25:03 明确说，基础模型能力会影响 Agent 架构。当模型能力短期内不进步时，为模型加脚手架通常有帮助：更强的检索、更细的规则、更固定的计划、更强的外部校验，都可能提升效果。因为弱模型本来做不到，外部机制替它做一部分工作。

但当模型能力进入下一层级，过强的脚手架会产生反效果。讲者拿 RAG 和 Claude Code 的 `grep/glob` 探索作对照：RAG 是人类预定义的检索范式，它先把相关片段找出来，再交给模型；而强模型有机会通过熟悉的搜索工具逐步探索项目结构，在搜索过程中学习代码库。

这里还有一个关键前提：第一方模型厂商可以围绕工具使用做后训练。Claude Code 式的 `grep` 与 `glob` 路线能变得更稳，一方面来自工具简单、接近开发者日常习惯，另一方面来自模型侧可以针对常见 Linux 原语和搜索行为继续训练。架构简化没有把复杂度凭空抹掉，一部分复杂度被转移到模型能力和训练分布里。对 Agent 团队来说，每次基础模型升级后，都要重新评估哪些外部脚手架还必要，哪些能力已经可以交还给模型。

## 脚手架的工程含义

脚手架是为了让当前模型完成原本难以完成的工作，例如固定检索流程、固定计划模板、固定角色分工、固定总结格式。它像施工现场的支架：建设早期很必要，但楼体足够稳定后，支架继续留在核心动线上就会挡路。

这也解释了“渐进式暴露”为什么重要。强模型不一定需要系统一次性把所有材料塞进上下文。更好的方式可能是给它熟悉、可组合、低成本的探索工具，让它先看目录，再看文件，再搜索符号，再读取局部实现。信息一点点暴露，模型在每一步都能根据新观察调整策略。

对 Agent 团队来说，这里有一个很现实的产品问题：每次新模型发布，是否需要重新评估架构。讲者在 00:25:56–00:26:06 的问题非常尖锐：如果看到下一代模型能力进步，Agent 架构要怎么变？这个问题直接影响产品工程，并会落到旧流程是否还值得维护、工具表面是否过大、缓存是否被破坏、模型是否被过度指挥。

### 6.3 人类预定义范式要适时退场

讲者把 RAG 作为“人类范式”的例子。弱模型时代，RAG 很合理：模型检索能力、长上下文处理能力和代码导航能力有限，人类就设计一个外部管道，先召回片段，再让模型回答。这相当于把“找资料”这一步从模型手里拿出来。

到了强模型时代，情况会变化。模型可以自己思考、自己检索、自己根据工具反馈修正下一步。此时继续强行套一个固定 RAG 管道，就可能替模型做太多事，甚至把模型带到错误的局部最优。尤其在代码库里，相关性来自文本相似度，也来自调用路径、测试失败、目录结构、最近修改和命名约定。

#### RAG 的过期风险

RAG 当然有价值。风险在于把它当成永远正确的 Agent 架构。若模型已经能通过 `grep/glob`、文件读取和测试反馈逐步建立项目理解，固定召回片段可能会削弱探索过程，把代码库压扁成几段相似文本。

这里和 S4 的观点相互扣合：代码 Agent 的搜索过程本身会产生理解。一个开发者刚接手陌生仓库时，也很少只看别人摘出来的三段代码就开改；他会看目录、看入口、看测试、看错误堆栈、看相邻实现。强模型越接近这种探索方式，Agent 架构越应该给它空间。

### 6.4 Chain-of-Thought 可能只是训练轮

讲者随后提出一个更有争议也更值得谨慎表达的思想实验：Chain-of-Thought，简称 CoT，是否也是一种人类预定义的辅助轮？他提到 DeepSeek-R1、Kimi K1.5 这类 thinking model 的训练思路：围绕问题生成大量 CoT，再把这些推理轨迹训练回模型，让模型获得 thinking 能力。

CoT 的直觉很像教小孩做题：一步一步写过程，降低犯错概率。对当前模型来说，这确实有效。问题是，如果未来模型能力继续增强，人类规定的“必须一步一步显式展开”会不会逐渐退场？讲者的回答很谨慎：他倾向于认为 CoT 属于人类预定义范式，未来可能消失，但什么时候消失不好说。

#### 不要把 CoT 消失写成硬预测

这一段应当理解为思想实验。讲者没有给出时间表，也没有说 CoT 已经失效。他真正强调的是：任何人类设计的推理脚手架都要接受模型能力变化的检验，CoT 也不例外。

更准确的工程表述是：CoT 有两层价值。第一层是外显推理轨迹，方便训练、监督、调试和人类阅读；第二层是模型内部获得的推理能力。未来如果模型能在内部完成更可靠的推理，外显 CoT 的产品形态可能变化。但在当前阶段，CoT 仍然是很多复杂任务、数学任务和可解释调试的重要工具。

## 6.5 回到第一性原理：让模型自由观察、试错、反馈

讲者在 00:27:24–00:28:03 给出架构建议：减少硬编码规则和特例，回到当前模型能力下最简洁、最第一性原理的 Agent 架构。这里的“第一性原理”有具体工程含义，它指向几个动作：观察环境、搜索信息、尝试执行、接收反馈、再调整策略。

这种架构的目标是获得模型能力的“溢出”。当新模型更会搜索、更会读代码、更会处理长任务时，简洁架构能让这些能力自然表现出来。反过来，如果旧系统把每一步都锁死，新模型来了也只能按旧流程走，能力提升很难转化为产品提升。

### 简洁架构的判断标准

一个 Agent 架构是否简洁，不能只看模块数量。关键要看它是否方便模型理解，是否保留必要反馈环，是否允许模型根据观察调整行动，是否减少对旧模型弱点的永久绑定。

这也解释了为什么讲者没有走向“什么都不用设计”。他批评的是过度硬编码，保留的是必要架构。Agent 仍然需要工具、上下文、边界、权限、测试和成本控制。区别在于，这些设计应围绕模型如何更好地观察和行动展开，不能把模型当成只会填空的弱组件。

## 6.6 ReAct：短期内最难被拿掉的核心循环

讲者在 00:28:17–00:28:33 回到 ReAct，认为各种方案和 trick 最后都会回到它最核心的部分：观察、思考、执行、再观察。这个循环短期内不会改变，因为 Agent 只要处理真实世界或真实代码库，就必须从环境获得反馈。

ReAct 可以写成一个最小循环：

$$O_t \rightarrow r_t \rightarrow a_t \rightarrow O_{t+1}$$

符号含义如下：

- $o_t$ ：第  $t$  步观察到的环境状态，例如文件内容、命令输出、错误日志、用户反馈。
- $r_t$ ：模型在当前观察基础上的推理或计划。
- $a_t$ ：模型选择的行动，例如调用工具、修改文件、运行测试、提出问题。
- $o_{t+1}$ ：行动之后得到的新观察。

这个循环之所以耐用，是因为它抓住了 Agent 区别于单轮问答的核心：行动会改变世界，世界会返回新证据，新证据会改变下一步行动。RAG、Subagent、tool design、memory、CLI 都可以看作围绕这个循环的不同增强方式。

## 6.7 本章小结

本章围绕 00:24:04–00:28:41 的 Bitter Lesson 展开，讲者给 Agent 架构提出了一个硬判断：模型能力变强时，架构要重新评估，很多为弱模型准备的复杂机制会逐渐失去优势。

## Bitter Lesson for Agent

- 基座模型的能力边界影响 Agent 架构
- 人类的预定义范式要适时的进入/退出
- 不靠大量硬编码规则和特例，而靠搜索、试错、反馈闭环
- 架构简洁，不断调整最核心的策略，方便模型理解
- 最核心的依旧是 ReAct 中的，观测 -> 思考 -> 执行 -> 观测

<https://www.incompleteideas.net/InIdeas/BitterLesson.html>

图 11: Bitter Lesson 在 Agent 架构上的落点：基座模型能力边界决定脚手架的多少，人类预定义范式需要适时退出，最终仍保留观察、思考、执行、再观察的 ReAct 最小循环。

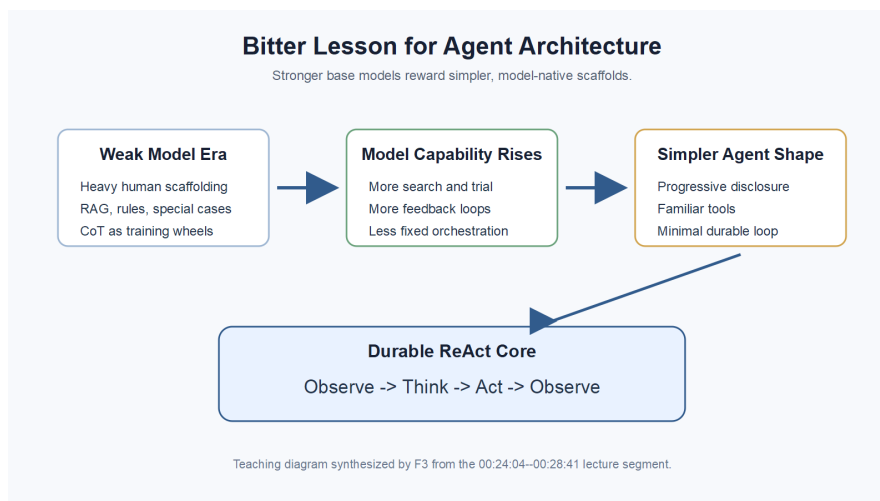


图 12: F3 重绘的教学示意图：当模型能力上升，工程重心从厚重脚手架转向渐进暴露、熟悉工具与可反馈的闭环，ReAct 循环成为较稳定的核心骨架。

- Bitter Lesson 提醒工程团队：短期有效的复杂技巧，未必能长期吃到算力和模型增长的红利。
- base model capability 会改变 Agent architecture。模型弱时需要更多脚手架，模型强时过多脚手架会拖累执行。
- RAG、固定计划、显式 CoT 等人类预定义范式都要接受同一个问题：它们是在帮助当前模型，还是在限制下一代模型。
- CoT 的退场只能作为思想实验处理，时间不确定，当前仍有训练、调试和复杂推理价值。
- ReAct 的观察、思考、执行、再观察循环仍是短期内最稳的 Agent 最小核心。

这一章为后续工具设计埋下伏笔。既然架构应当尽量简洁、模型友好、能吃到能力增长红利，那么工具也不能只按开发者视角设计。下一章会进一步讨论：工具名、工具描述、工具数量和 CLI 的渐进式暴露，本质上都是 prompt design 的一部分。

## 7 工具设计与 CLI 趋势：工具也是提示词

前一章用 Bitter Lesson 提醒读者：当基础模型能力变强时，Agent 架构应当尽量贴近模型已经会用、已经被训练过的形式。本章把这个原则落到工具层。演讲在 00:28:41 之后的核心判断很直接：工具调用没有魔法，工具结果也没有魔法，它们最终都会变成上下文里的文本。只要进入模型上下文，工具名、参数名、描述、返回值、错误信息和系统提醒就共同构成了提示词。

这句话听起来像工程常识，实际会改变工具设计重心。很多团队把工具当成给开发者调用的函数：函数名便于代码阅读，参数名便于 IDE 补全，返回值便于程序解析。Agent 场景里，工具首先要被模型理解。模型看到的是序列化后的说明文字和工具回传文本。前文 S2 已经说明过 OpenAI Harmony 这类格式如何把结构化 API 内容变成带标签的模型输入；这里同理，tool call 和 tool callback 只是上下文里的特殊片段。

### 工具没有特权通道

工具调用的可靠性来自模型训练、上下文格式和约束机制的共同作用。工程上不能假设“这是 tool callback，所以模型一定只按工具结果理解它”。更稳妥的心智模型是：凡是塞进上下文的文本，模型都会把它纳入下一步判断。

### 7.1 工具回传也是上下文：没有高低贵贱

演讲在 00:28:44–00:29:49 先讲了一个观察：某些版本的 Claude Code 曾经会把 system reminder 一类提醒塞进任意一个 tool callback 或 return message 后面。这个现象有教学价值，因为它说明 callback 并没有不可侵犯的语义边界。只要文本出现在上下文里，模型就可能读取、理解并执行其中的提醒。

这也解释了为什么工具日志、错误堆栈和中间结果不能被视为“机器内部噪声”。对模型来说，它们和用户消息、开发者提示词、工具描述一样，都在参与下一轮推理。前文 S4 讨论过“失败工具调用要不要保留”的两难：删掉失败记录能省 token，也可能让模型重复踩坑。这里的工具视角进一步说明原因：失败记录本身是一种上下文证据，它告诉模型“这条路刚才走不通”。

可以用一个轻量公式描述工具上下文的风险。设一次 Agent 调用的有效上下文为：

$$X = P_{\text{sys}} + P_{\text{dev}} + U + T_{\text{schema}} + T_{\text{callback}} + H$$



其中每一项都可能影响下一步行为。

- $P_{\text{sys}}$ : system prompt, 通常承载稳定规则。
- $P_{\text{dev}}$ : developer prompt, 承载开发者级约束。
- $U$ : 用户输入和任务目标。
- $T_{\text{schema}}$ : 工具名称、描述、参数 schema 等工具元数据。
- $T_{\text{callback}}$ : 工具回传结果, 包括成功输出、错误信息和中间数据。
- $H$ : 历史消息、计划、提醒、压缩摘要等其他上下文。

这个公式的意思很朴素: 工具只是  $X$  的一部分。工具结果如果冗长、模糊、带有歧义, 模型下一步就会在一个更脏的  $X$  上行动; 工具结果如果稳定、清晰、贴近模型已有知识, 下一步就更容易对齐。

## 7.2 工具名、描述和参数名都在写 prompt

00:29:50–00:30:30 的第二个判断是: 工具设计要面向模型。很多 Agent 工具失败, 根因常在工具说明: 模型无法判断何时调用、如何填参、调用后该期待什么结果; 底层 API 本身可能完全可用。

这与传统函数命名不同。传统软件里的函数名主要服务人类维护者, 足够短、符合项目习惯即可。Agent 工具名则更像一个微型 instruction。比如浏览器类工具统一以 **browser** 开头, 计算机控制类工具统一以 **computer** 开头, 这种命名纪律会给模型一个清晰的类别信号。模型不需要在一堆风格各异的名字中猜测工具边界。

工具 schema 也一样。参数名如果写成 `x`、`payload`、`data`, 开发者当然知道含义, 模型却会失去语义锚点。更好的参数名应当把任务意图暴露出来, 例如 `query`、`path`、`selector`、`max_results`。这些词具有实际提示作用: 它们会进入模型上下文, 成为模型选择参数的依据。

### 工具 schema 的三层提示

工具 schema 至少承担三层提示作用: 第一, 工具名提示“何时用”; 第二, 工具描述提示“为什么用、用完得到什么”; 第三, 参数名和类型提示“怎样填”。这三层提示互相矛盾时, 模型会靠猜, 调用成功率自然下降。

## 7.3 把结构化输出伪装成工具调用

演讲在 00:30:33–00:31:59 给了一个很实用的例子: 如果任务是给文本打标签, 很多人的第一反应是要求模型“只输出标签, 不要输出其他文本”, 随后再用正则表达式解析。这条路能跑通, 但它把结构化约束放在了自然语言里, 后处理还要承担兜底工作。

另一种方式是定义一个专门的打标工具。工具的效果就是“提交标签”, 参数就是候选标签或标签字段, 然后要求模型每次返回时调用这个工具。这样做利用了模型厂商在 tool call 正确率上的训练和推理约束。演讲中提到的“限制采样”可理解为 constrained sampling: 模型生成工具调用时, 解码过程会更强烈地受 schema 约束, 参数类型和字段结构更容易保持正确。

这个技巧的关键点在于: 表面上只是多定义了一个工具, 实际收益来自把原本脆弱的自然语言输出约束, 迁移到模型更擅长遵循的工具调用通道里。对分类、标签、结构化选择、路由决策这类任务尤其有用。

## 结构化输出的实战建议

当输出空间有限、字段结构明确、错误解析代价较高时，可以优先考虑用 tool call 承载结构化结果。它通常比“请只输出 JSON”或“请只输出标签”更稳，因为模型在工具调用场景中受到更强的 schema 约束。

### 7.4 不要标新立异：模型也吃“通识”

00:32:01–00:33:01 的观点延续了 S4 对 RAG 与 grep/glob 的讨论。演讲把模型比作一个正在读书的大学生：它有通识，也有训练中反复见过的工具习惯。一个工具越接近模型已经熟悉的人类工具，模型越容易理解它的使用时机。

这就是 grep、glob 这类原语的优势。它们在大量代码、文档、问答和教程里出现过，模型对“搜索文本”“按模式找文件”“逐层探索项目”的语义很熟悉。相反，如果团队设计了一个全新的 RAG 工具、一个内部 DSL 或一个很复杂的检索接口，就需要在 prompt 里解释它是什么、怎样调用、什么时候不该调用、参数如何影响结果。连人类都要读半天文档才能理解的工具，交给模型后通常会出现三类问题：调用错工具、填错参数、在错误时机调用。

这里的反直觉点在于：高级抽象不一定让 Agent 更聪明。高级抽象省掉了探索过程，也可能省掉了模型建立项目理解的过程。S4 已经强调，搜索过程本身会教会模型仓库结构；本章强调，搜索工具本身也要让模型“似曾相识”。

### 7.5 工具数量、MCP 与缓存压力

工具面太宽会立刻撞上 S2 的 prompt cache 问题。每增加一批工具，模型调用前缀里就多出一批工具名、描述和 JSON Schema。若不同用户、不同项目、不同 MCP 服务器暴露的工具集合经常变化，prompt cache 的稳定前缀就被破坏，成本和延迟都会上升。演讲在 00:33:03–00:34:19 说得很直：工具太多会影响性能，MCP 火过一阵后，大家也开始意识到它带来的 cache 与性能问题。

可以把工具面成本写成一个简单直观式：

$$E[C_{\text{agent}}] \approx p_{\text{hit}} C_{\text{hit}} + (1 - p_{\text{hit}}) C_{\text{miss}} + C_{\text{tool-surface}}$$

- $p_{\text{hit}}$ ：prompt cache 命中概率。
- $C_{\text{hit}}$ ：缓存命中时的成本和 prefill 延迟。
- $C_{\text{miss}}$ ：缓存未命中时的成本和 prefill 延迟，通常远高于命中状态。
- $C_{\text{tool-surface}}$ ：工具说明、schema、回传结果和工具选择带来的额外上下文成本。

这个式子不承担严格计费功能，它承担工程判断功能：工具越多，schema 越长，回传越重，缓存越不稳定，Agent 的成本曲线越难看。

演讲提到两类补救办法。第一类是 tool search，也就是把一部分工具延迟暴露，让模型先搜索可用工具，再决定调用。这里的具体产品名在 ASR 中不完全确定，需以截图为准；本章按语义使用 tool search。第二类是 scripted tool，例如让模型写一个工具调用列表或脚本，把多次读取 Excel、查询数据、处理中间结果的动作包在脚本里执行，避免每一步中间结果都塞回上下文。

这两类办法有价值，但演讲者也提醒它们带有 hack 感：它们是在工具面已经太大、上下文已经太重之后做补丁。更根本的问题仍然是：为什么一开始需要暴露这么多工具？

## 7.6 CLI 的优势：渐进式披露

00:34:19–00:34:56 的最后判断是本章的落点：未来更稳的方向可能是 CLI，特别是渐进式披露的 CLI。这里的 CLI 代表一种上下文管理策略，复古情怀只占很小一部分。

人类使用 CLI 时通常不会把完整手册背下来。我们先知道一个命令的大概用途，遇到不确定时再运行 `-h`、`--help` 或查看文档。模型也可以采用类似路径：初始上下文只暴露最小接口和简短说明；当它需要更细的参数、示例或边界条件时，再主动查询帮助。这比一次性把所有 MCP 工具和长 schema 塞进上下文更符合渐进式披露原则。

### 为什么 CLI 适合 Agent

CLI 把工具知识拆成多层：命令名是入口，简短 help 是第一层说明，详细文档是第二层说明，实际运行结果是反馈。模型可以按需展开知识，避免一开始就背负庞大的工具说明。

这也回应了 S6 的 Bitter Lesson：随着模型能力增强，工程系统应减少精致但脆弱的中间结构，把更多选择权交给模型，同时提供足够清晰、足够熟悉、可逐步展开的操作界面。CLI 正好满足这几个条件：文本化、可组合、可查询、训练语料中常见，并且天然支持一步一步探索。

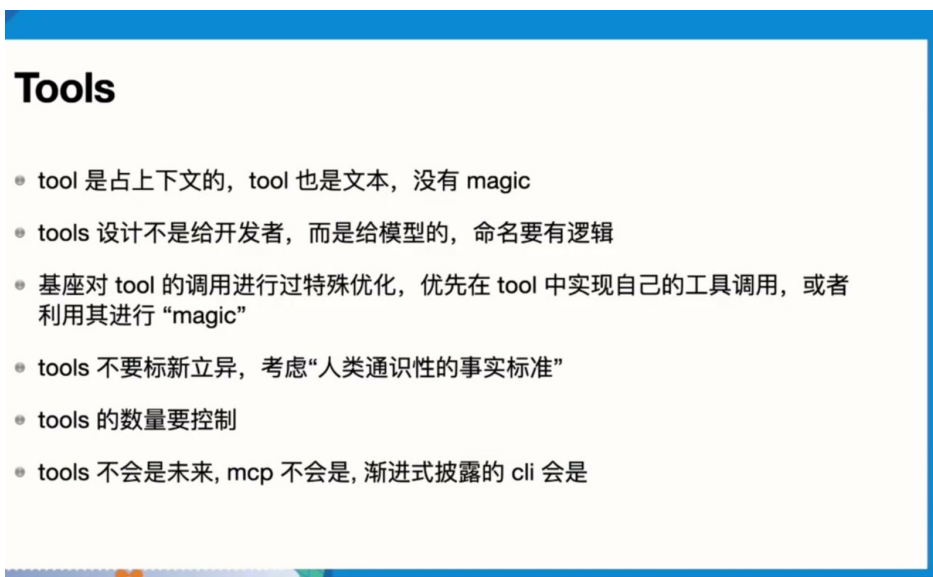


图 13: 工具设计的核心约束：工具调用、参数、回传文本都会进入上下文；工具面过大还会牵动缓存、性能与 CLI 渐进披露策略。

## 7.7 本章小结

本章把“上下文是一切”推进到工具层。tool call、tool callback、tool schema、工具名、参数名、错误信息和系统提醒都会成为模型下一步行动的依据。工具设计因此不能只按开发者函数接口来做，它必须按 prompt 来做。

实战上有四条可执行原则。第一，工具结果要被当成上下文证据管理，失败记录和中间结果是否保留要结合 benchmark 验证。第二，工具命名、描述和参数 schema 要面向模型理解，保持类别一致和语义清晰。第三，结构化输出可优先借助 tool call 和 constrained sampling 提高可靠性。第四，工具面要克制，过多 MCP 和长 schema 会伤害 cache、成本、延迟和性能。

本章最终落在 CLI：渐进式披露让模型像人类一样先读少量说明，再按需查询更多细节。它把工具设计从“全量暴露”拉回“按需学习”，也为下一章的 memory 问题埋下伏笔：Agent 到底应该在什

么时候记住信息、检索信息、暴露信息？

## 8 Memory 与市场教育：长程智能体如何被接受

工具面讨论之后，演讲在 00:34:56 进入 memory。这个转场很自然：工具回答“模型现在可以调用什么”，memory 回答“模型以后应该记住什么”。两者都会进入上下文，也都会带来成本、延迟、误激活和信任问题。

本章的语气需要保持克制。演讲者没有给出一个成熟的 memory 方法论，反而明确说大家都还在探索。Claude Code 近期才开始支持 memory，这本身就说明问题没有被彻底解决。把 memory 说成“给 Agent 加一个长期记忆模块”很容易，真正难的是三件事：什么值得记、什么时候取出来、旧记忆怎样更新。

### 8.1 Memory 的三道难题：抽取、检索、更新

00:35:00–00:35:43 的第一组问题集中在 memory 的生命周期。模型并不稳定知道哪些信息应该被记住。一次对话里可能有用户偏好、项目约定、临时假设、错误路径、外部环境、依赖版本和过期计划。人类听完会自然区分“长期偏好”和“这次任务里的临时条件”，模型却经常把二者混在一起。

第一道难题是抽取。抽取和对话摘要追求不同目标。摘要追求覆盖，memory 追求可复用。比如“用户今天想修一个测试”通常不该长期保存；“这个仓库统一用 pnpm，禁止 npm install”可能值得保存；“某个临时 API 现在报 500”只在短期上下文有用。抽错了，长期记忆会变成噪声。

第二道难题是检索。很多人会自然想到语义检索或 RAG：把 memory 切块、向量化、按问题召回。演讲者在 00:35:43–00:36:03 给了一个更直白的观察：很多场景下，把所有 memory 放进一个模型调用的上下文里，再让一个小模型结合当前问题挑选，检索效果反而最好。原因不难理解：memory 的相关性经常依赖任务语境，纯向量相似度只能看到字面或语义距离，模型阅读全量 memory 后能做更细的条件判断。

第三道难题是更新。代码仓库会变，依赖库会变，打包工具会变，团队约定也会变。memory 如果不更新，就会把旧世界带进新任务；memory 如果每次都调用一个大 Agent 重新整理，又会和“用 memory 减少算力”的初衷冲突。演讲者称这个问题“得不偿失”，意思是：记忆系统如果维护成本高到抵消收益，就只是把复杂度挪了个地方。

#### Memory 的失败形态

坏 memory 的危害会超过“没帮上忙”。它会主动污染上下文：把过期依赖当成当前事实，把临时偏好当成长期偏好，把一次失败路径当成团队规范。长期智能体会遗忘，更糟的是自信地记错。

### 8.2 全量上下文筛选与语义检索的取舍

演讲中“把所有 memory 放进上下文，用小模型做结合”的建议，听起来与前文控制上下文长度的原则有张力。这个张力正是 memory 工程的真实处境。语义检索便宜、快速、可扩展，但可能召回不到真正相关的记忆；全量上下文筛选更准，却消耗上下文窗口和调用成本。

可以用一个简化目标来理解两种方案：

$$Q_{\text{memory}} = R_{\text{relevant}} - R_{\text{noise}} - C_{\text{read}} - C_{\text{update}}$$

- $Q_{\text{memory}}$ ：memory 对当前任务的净贡献。



- $R_{\text{relevant}}$ : 相关记忆带来的收益。
- $R_{\text{noise}}$ : 无关或过期记忆带来的干扰。
- $C_{\text{read}}$ : 读取、筛选、注入 memory 的成本和延迟。
- $C_{\text{update}}$ : 维护、清理、改写 memory 的成本。

工程目标应让  $Q_{\text{memory}}$  为正。很多产品一开始追求“记得更多”，很快会遇到反噬：上下文更长、cache 更差、模型更容易被旧信息牵着走。更稳的做法是先定义少量高价值记忆类型，例如用户长期偏好、仓库稳定约定、工具链固定入口、反复确认过的业务规则，再逐步扩展。

### 8.3 system prompt 里的记忆会过度激活

00:36:15–00:36:47 的一个细节非常重要：把 memory 放进 system prompt 会让模型过度激活这些信息。用户可能只想聊一个新问题，模型却突然提起“你上次问过什么”。这类打扰通常来自 system prompt 的高优先级地位：它暗示模型这些信息很重要，最好在回答中体现。

这会带来两个副作用。第一，无关记忆会抢占注意力。模型在生成答案时会努力解释或利用 system prompt 中的内容，哪怕它与当前问题关系很弱。第二，发散性思维会下降。演讲者说，如果 full prompt 放太多，模型会“变懒”，这可以理解为：上下文给了太多既定轨道，模型更倾向沿着已有记忆走，减少重新探索。

这与 S3 的长上下文退化、S4 的压缩幻觉和 S7 的工具面膨胀是同一个问题的不同侧面。上下文越满，收益越可能被噪声抵消。越多的“可能有用信息”会形成更强的先验，让模型更难判断当前任务真正需要什么。

#### 不要把 memory 当成永久系统法条

system prompt 适合放稳定、少量、强约束的规则。memory 往往带有时效性和场景性，直接塞入 system prompt 容易让模型过度引用。更合理的方式是按任务筛选后注入，或让模型在需要时显式读取。

### 8.4 memory.md 与 JSON memory

演讲在 00:36:47–00:37:14 提到两类常见存储方案：memory.md 和 JSON memory。二者都不神秘，差别在于对结构和可编辑性的取舍。

memory.md 把记忆当成一份可读可改的 Markdown 文档。它的优点是贴近模型训练语料，也贴近人类维护习惯。模型可以直接阅读段落、标题和列表，人类也能快速审查、删除或改写。它放弃了一部分结构化能力，换来更低的理解门槛。

JSON memory 则把记忆拆成结构化字段，例如偏好、项目约定、工具链、时间戳、置信度、来源等。它更适合程序化筛选、排序和更新，也更适合做权限控制或审计。但 JSON 的字段设计本身又会变成一套 schema 工程：字段太少，表达力不足；字段太多，模型写入和维护会变复杂。

两种方案没有天然高下。若团队还在探索 memory 类型，memory.md 更适合快速迭代；若已有稳定记忆类别和明确更新流程，JSON memory 更便于治理。关键仍然是 benchmark：memory 是否提升目标任务，是否降低返工，是否减少用户重复解释，是否增加误激活。



## 8.5 市场教育：用户开始接受看结果

00:37:14–00:38:47 是演讲的实质性收尾。演讲者围绕 Claude Code/Cursor/Manus 这一类长程 Agent 产品，强调其市场教育价值；其中 Claude Code 一词来自上下文校正，画面标题仍有 Open-Claw/Claw 的不确定性。2025 年初，很多讨论还在担心算力溢出：没有那么多需要和大模型 chat。随后编码 Agent 和通用 Agent 产品改变了用户预期。用户开始接受把更长链路的任务交给模型，减少对每个中间结果的逐步检查，直接看更高层次的结果。

这句话很重。早期人与模型协作像结对编程：模型写一步，人类看一步；模型给中间结果，人类确认后再继续。Claude Code、Cursor、Manus 等产品训练用户接受另一种关系：给 Agent 一个目标，让它自己搜索、编辑、运行、修复，最后交付结果。DeepSeek 在中国市场的普及也起到类似教育作用，让更广泛的人群认识到大模型可以进入真实任务链路。

这种市场教育释放了长程智能体的想象力。用户愿意给 Agent 一台电脑、一个仓库、一段时间和一个目标，让它自己完成多步工作。这种信任一旦形成，工程责任也同步上升。用户不看每一步，并不意味着每一步可以随便做；用户只看结果，意味着系统必须在上下文、工具、memory、成本、延迟和回滚能力上更可靠。

讲者把这个变化说得更极端：用户可能只给 Agent 一台电脑和一个目标，过程中写了多少代码、改了多少文件都不逐步检查；程序运行失败时，也期望 Agent 自己定位并修复。这个例子强调验收重心上移，同时提醒团队保持警惕：用户从盯每一步动作，转向关注最终结果、失败恢复能力和系统能否自证可用。

### 市场教育的真正含义

市场教育的含义在于：用户愿意把注意力从每个中间动作移到最终结果、验收标准和失败恢复能力上，同时保留对“模型永远正确”的警惕。Agent 产品因此从聊天工具变成任务执行系统。

## 8.6 长程智能体的信任边界

一旦用户接受长程 Agent，memory 的意义也变了。短对话里，memory 只是减少重复说明；长程任务里，memory 决定 Agent 是否能延续项目约定、尊重用户偏好、记住已验证过的失败路径，并在多轮任务中保持一致。

但信任不能靠口号建立。一个可托付的长程 Agent 至少需要四类边界。第一，记忆边界：什么会被记住，什么只在当前任务有效。第二，工具边界：Agent 能调用哪些工具，哪些操作需要用户批准。第三，验证边界：Agent 如何证明结果可用，例如运行测试、给出日志、标出未验证部分。第四，成本边界：长任务会花多少 token、时间和外部资源。

这些边界连接回整场演讲的主线。上下文构造决定模型看到什么；benchmark 决定团队如何衡量改进；架构简化决定系统是否能跟上基础模型能力；工具设计决定模型怎样行动；memory 决定 Agent 如何跨任务延续经验；市场教育决定用户愿意把多长的链路交出去。

## 8.7 本章小结

Memory 仍是 Agent 工程中未完全解决的问题。真正的难点集中在抽取、检索、更新和误激活，而非简单存下一段文字。全量 memory 上下文筛选可能比朴素语义检索更准，但会带来成本和工具面压力；把 memory 放进 system prompt 会强化无关记忆，限制模型发散；memory.md 和 JSON memory 分别代表可读可改与结构化治理两种路线。

本章最后的市场教育观点是全讲的收束：Claude Code、Cursor、Manus 和 DeepSeek 类产品让用户开始接受长程 Agent，愿意用最终结果而非每个中间步骤来判断系统价值。这种接受会抬高工程

## Memory

- claude code 在最近才开始支持 memory
- 模型并不知道什么是应该记住的，也很难正确提取关键点
- memory 如何存储，如何分层次存储，如何检索，如何读取，如何更新
- memory 对发散性思维的限制
- 常见的两种解决方案
  - memory.md

图 14: Memory 的工程难点集中在抽取、存储、检索、读取、更新和副作用控制：记忆一旦长期暴露给系统提示词，模型可能被无关信息过度激活。

## OpenClaw

- OpenClaw 的价值在于市场教育
- 回到 2025 年初，cursor、claude code、manus 完成了对市场智能体的教育
- DeepSeek 对中国市场的教育
- Claw 让大家放开了对全自主长存智能体的思维，也教育了市场的接受度

图 15: 市场教育让用户逐步接受长链路 Agent：关注更高层结果，放宽对每一步中间过程的人工检查。

标准。越是用户不逐步盯着看，Agent 越要在上下文、工具、记忆、验证和成本上经得起检验。

## 9 最终综合：Agent 工程的三条主线

这场分享的标题是“Agent 一线实战：落地经验与 Know-How”。它没有试图建立一套封闭方法论，演讲者也反复提醒，Agent 还在高速变化，很难现在就给出定式。真正贯穿全场的，是一线工程团队在反复试错后形成的三条主线：上下文主线、评测主线、架构主线。最后关于 Claude Code、Cursor、Manus 与 DeepSeek 的市场教育讨论，则把这三条主线推向一个更现实的问题：当用户开始信任长程 Agent，工程系统必须承担更高的结果责任。



图 16: 讲义综合图，由 S7-S8 与全课主线合成：上下文、评测、架构和信任共同约束可落地的 Agent 产品。

### 9.1 第一条主线：一切进入模型的东西都是上下文

S2 已经定义过上下文：system prompt、developer prompt、用户消息、工具 schema、图片、音频、tool callback、错误日志、动态提醒，最终都会变成模型可读的输入。S7 再把这个判断推进一步：工具也只是上下文。S8 又补上一层：memory 也只是上下文的一种长期来源。

这条主线的工程含义很硬。Agent 质量同时受模型参数和上下文构造影响。把什么放进去、放在哪里、用什么顺序、保留多久、何时压缩、何时删除失败记录、何时读取 memory，都会改变模型下一步行为。上下文像战场地图：地图画错了，再强的执行者也会沿着错误路线前进。

因此，Agent 工程的第一项基本功应当落在上下文纪律上，框架包装只能排在后面。稳定内容要帮助 prompt cache 命中，动态内容要避免污染稳定前缀；搜索过程可能比搜索结果更有价值；失败记录可能浪费 token，也可能防止模型重复犯错；memory 可能减少重复说明，也可能把旧事实强行带回当前任务。

### 9.2 第二条主线：没有应用 benchmark，就没有真实改进

S3 的 benchmark 讨论是全场最像工程管理准则的一段。模型版本会变，供应商会更新，系统 prompt 会调整，工具集合会扩张，长上下文会带来能力下降。若团队只看官方榜单或一次肉眼体验，

很容易把随机波动当成改进，把场景退化当成“模型偶发失误”。

应用 benchmark 的价值在于建立产品自己的地面标尺。演讲者建议同一 benchmark 至少跑五遍、十遍来平均模型不稳定性，这一点非常关键。大模型输出带有方差，单次成功无法证明系统稳定，单次失败也不能立刻说明路线错误。团队需要一个  $B_{app}$ ，也就是面向自身任务的基线分数，来比较新模型、便宜模型、新工具、新 memory 策略和新上下文压缩方式。

这条主线还解释了为什么很多 Agent 争论很难靠口头说服。RAG 与 grep/glob 谁更好，失败记录要不要保留，MCP 工具面要多大，memory 放 system prompt 还是按需读取，答案都依赖任务分布。没有 benchmark，讨论只是在交换偏好；有了 benchmark，团队才知道自己在优化什么。

### 9.3 第三条主线：模型变强时，架构要变简单

S6 借 Bitter Lesson 提醒：复杂的人造机制常常只在某个模型能力阶段有效。模型弱时，脚手架能补洞；模型强时，过重脚手架会限制模型。这个判断贯穿前后章节。Cursor 式 RAG 与 Claude Code 式探索的比较，Subagent 与 Agent Team 的比较，MCP 工具面与 CLI 渐进式披露的比较，本质上都在问同一个问题：哪些结构是在帮模型，哪些结构已经开始挡模型？

简单并不等于粗糙。ReAct 的观察、思考、行动、再观察仍然是最小核心；CLI 的渐进式披露也需要良好文档、清晰命令和可靠反馈。这里的“简单”指减少过早固定的中间范式，把更多判断交给模型，同时给模型熟悉、可组合、可验证的操作原语。

这条主线对团队有一个直接要求：架构不能只在诞生时设计一次。每次基础模型能力跃迁，团队都要重新评估旧机制。旧机制可能曾经是护栏，后来变成负重；旧 prompt 可能曾经提高准确率，后来压制模型发散；旧 memory 可能曾经节省沟通，后来积累过期偏见。

### 9.4 一个耦合系统：成本、延迟、工具、记忆与信任

三条主线分开讲很清楚，落地时会耦合在一起。工具多了，schema 变长，prompt cache 变差，prefill 延迟上升；为了减少工具面，团队引入 tool search，又增加一次搜索决策；为了让 Agent 长程工作，团队加入 memory，又带来检索、更新和误激活；为了证明结果可信，团队运行更多测试和校验，又增加工具调用与成本。

这件事并不轻松，它是 Agent 从 demo 进入产品后的真实形态。一个好的 Agent 团队不会孤立优化“聪明程度”，它会同时看五件事：上下文是否稳定，benchmark 是否反映目标任务，架构是否贴合当前模型能力，工具是否能被模型可靠理解，memory 是否真的带来净收益。最终还要加上一件更难的事：用户是否愿意信任这个系统交付的结果。

可以把全场综合成一个判断式：

$$V_{\text{agent}} \approx Q_{\text{context}} \times B_{\text{app}} \times A_{\text{fit}} \times T_{\text{trust}} - C_{\text{cost+latency}}$$

- $V_{\text{agent}}$ ：Agent 产品在真实任务里的有效价值。
- $Q_{\text{context}}$ ：上下文构造质量，包括 prompt、工具回传、搜索过程、压缩和 memory。
- $B_{\text{app}}$ ：应用 benchmark 所代表的场景基线与稳定性。
- $A_{\text{fit}}$ ：架构与当前模型能力的匹配程度。
- $T_{\text{trust}}$ ：用户对长程执行和最终结果的信任。
- $C_{\text{cost+latency}}$ ：成本、延迟、cache miss、工具调用和验证带来的总负担。

这个式子不承担数学定理功能，它是一条产品工程提醒：任何一项短板都会拖垮 Agent。上下文脏，模型会走偏；benchmark 缺失，团队会误判；架构过重，能力被压住；用户不信任，长程 Agent 无法被托付；成本和延迟失控，商业上跑不起来。

### 9.5 市场教育完成后，工程门槛更高

S8 结尾的市场教育观点值得放在最终综合里重新强调。Claude Code、Cursor、Manus 这类产品让用户接受“看更高层次的结果”：给 Agent 一个目标，让它自己搜索、编辑、运行、修复，最后交付结果。DeepSeek 对中国通用市场的教育也让更多用户相信大模型可以进入真实任务链路。

这改变了 Agent 工程的验收方式。用户减少对中间过程的逐步检查，系统就必须自己承担过程质量：知道何时查文档，何时保留失败记录，何时请求授权，何时更新 memory，何时停止继续烧 token，何时承认无法完成。

**全场最终判断**

Agent 的护城河来自一线 Know-How：知道模型在哪些上下文里会变好，在哪些工具面前会犯错，在哪些 benchmark 上才算真的进步，以及用户愿意把多长的任务链路交给它。

### 术语与符号速查

**Agent** 由模型驱动的任务循环，核心动作是观察状态、选择行动、接收结果、更新上下文。

**Context** 模型当轮可见的全部输入，包括 prompt、工具描述、工具回传、文件片段、图片、音频和动态提示。

**Benchmark / Baseline** 前者是代表真实任务的评测集合，后者是固定配置在该集合上的基线表现。

**Scaffold** 为弥补模型能力不足而加的外部脚手架，例如固定检索、固定计划、固定摘要和外部校验。

**ReAct** 观察、思考、行动、再观察的最小 Agent 循环，适合作为理解复杂 Agent 架构的底层模型。

**Memory** 跨会话保存并按需读回的信息层，难点在抽取、检索、更新和误激活控制。

$Q_{\text{context}}$  上下文构造质量，衡量模型看到的信息是否稳定、相关、低噪声。

$B_{\text{app}}$  应用侧评测基线，帮助团队把模型、prompt、工具和 memory 改动转成可比较结果。

$A_{\text{fit}}$  架构与当前基础模型能力的匹配程度，提醒团队随模型升级重估旧机制。

$T_{\text{trust}}$  用户对长程执行和最终交付的信任，决定 Agent 能承担多长的任务链路。